



ORACLE
NetSuite

SuiteTalk REST Web Services

2025.2

January 7, 2026



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Overview of SuiteTalk REST Web Services	1
Limitations of REST Web Services	2
Concurrency Governance and Session Management	2
REST Web Services Key Concepts	3
REST Web Services and Other Integration Options	4
SOAP Web Services vs. REST Web Services Operation Mapping	5
REST Web Services URL Schema and Account-Specific URLs	7
Prerequisites and Setup for REST Web Services	9
Setting Up Authentication	11
REST Web Services Request Processing	13
Sending an Asynchronous Request	13
Sending an Asynchronous Request Using an Idempotency Key	14
Polling for the Status of a Completed Asynchronous Request	14
Getting the Result of an Executed Request	15
REST Web Services Operations	16
GovernanceLimits Operation	16
ServerTime Operation	17
Using Postman with REST Web Services	18
Importing and Setting Up a Postman Environment	19
Importing the Postman Collection	20
Sending a Request From the Imported Collection	21
Tutorial: Using Postman with OAuth 2.0	21
Setting Up Token-Based Authentication (TBA) - Tutorial	32
Working with Resource Metadata	34
Getting Metadata	35
Working with OpenAPI 3.0 Metadata	36
Working with JSON Schema Metadata	40
Working with Records	43
The REST API Browser	43
NetSuite Record Structure	45
Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1	45
Creating a Record Instance	46
Getting a Record Instance	47
Format of Sublists and Subrecords	49
Format of Selects and References	49
Format of Multiselects	50
Format of Enumeration Values	50
Updating a Record Instance	51
Using the Upsert Operation	52
Deleting a Record Instance	52
Accessing Subresources in REST Web Services	52
Using External IDs	53
Using Datetime Fields	54
Executing Record Actions	55
Transforming Records	56
Working with the Pricing Sublist on Item Records	57
Using the REST Web Services SuiteScript Execution Context	58
Working with Sublists	59
Creating a Sublist	60
Updating a Sublist	61
Replacing a Sublist	63
Deleting a Sublist	67
Working with Subrecords	69

Record Filtering and Query	71
Listing All Record Instances	71
Record Collection Filtering	72
Executing SuiteQL Queries Through REST Web Services	74
Working with SuiteAnalytics Datasets in REST Web Services	75
Collection Paging	77
Error Handling and Logging in REST Web Services	80
Error Handling in REST Web Services	80
Using the REST Web Services Execution Log	83

Overview of SuiteTalk REST Web Services

The NetSuite REST web services provide an integration channel that extends the capabilities of SuiteTalk. REST web services provide a REST-based interface for interacting with NetSuite.

With REST web services, you can:

- Use CRUD (create, read, update, delete) operations to manage NetSuite records and move between records. For details, see [Working with Records](#).
- Get and process the API definition and record metadata. For details, see [Working with Resource Metadata](#).
- Run NetSuite queries on records. For details, see [Record Filtering and Query](#).

REST Web Services Features

The following table provides an overview of features in REST web services, and how they work with records.

For information about the different REST web services features, see [Prerequisites and Setup for REST Web Services](#).

SuiteTalk REST Web Services	Generally Available Records
REST Record Service	Fully supported.
REST Query Service	Fully supported. To see which records you can use in queries, see the Records Catalog. For information, see the help topic Records Catalog Overview .

REST Web Services Key Features

Here's an overview of the main REST web services features:

- Metadata – for more information, see [Working with Resource Metadata](#).
- CRUD operations – for more information, see [Working with Records](#).
- Record Collection Filtering – for more information, see [Record Collection Filtering](#).
- Record Actions – for more information, see [Executing Record Actions](#).
- Record Transformation – for more information, see [Transforming Records](#).
- Asynchronous request processing – for more information, see [REST Web Services Request Processing](#).
- Workbook Dataset execution – for more information, see [Working with SuiteAnalytics Datasets in REST Web Services](#).
- Workbook Dataset List – for more information, see [Getting a List of Datasets Through REST Web Services](#).
- SuiteQL Query execution – for more information, see [Executing SuiteQL Queries Through REST Web Services](#).

Benefits of REST Web Services

The benefits of REST web services include:

- Simple access to records metadata. This includes user and company-specific metadata. For more information about working with records metadata, see [Working with Resource Metadata](#).
- Easier handling of custom records and custom fields.
- Easy to navigate API.
- Unlike RESTlets, you don't need to write, deploy, and run custom scripts.

Limitations of REST Web Services

Keep the following in mind when working with REST web services.

- Queries only return record IDs and HATEOAS links. (See [HATEOAS](#).) That is, query results have a form of non-expanded references. Also, you can only use body fields in query conditions.
- Translations in REST web services work the same way as in SuiteScript. For custom records, a user's language preference (Home > Set Preferences > Language) is taken into account. For standard records, translations are returned and updated in the company's primary language. Additionally, in the current version of REST web services, it is not possible to read and update translations on the translation sublist for both custom records and standard records. This behavior is different from the behavior of SuiteScript.
- REST web services don't support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).
- Asynchronous request execution isn't supported in the REST query service.
- REST web services always load record instances in edit mode, even for GET requests. When a user without the Administrator role tries to run a GET request for an employee record of a user with the Administrator role, this error message will appear: "For security reasons, only an administrator is allowed to edit an administrator record". This prevents users without the Administrator role from editing the employee record of a user with the Administrator role.

To save \n and \r strings through REST you need to escape the strings into \\n and \\r.

- Custom sublists are not available in REST web services.

Concurrency Governance and Session Management

To optimize NetSuite application and database servers, the system employs certain mechanisms to control the consumption of web services.

Concurrency for REST web services is governed by an account limit, where each request counts toward the account limit. The account governance limit applies to the combined total of web services and RESTlet requests. For detailed information about concurrency governance, see the help topic [Web Services and RESTlet Concurrency Governance](#).

Additionally, if a request takes more than 15 minutes to complete, it automatically times out.

These mechanisms ensure the following:

- Requests are monitored and controlled to ensure that the user experience is not excessively impacted.
- The burden of heavy web services users is not shared among all users.

You can use APM to monitor your REST web services integration's concurrency usage. For information, see the help topic [Monitoring Web Services and RESTlet Concurrency](#).

REST Web Services Key Concepts

The following sections introduce the key concepts of REST web services.

HATEOAS

Hypermedia as the Engine of Application State is a key principle that should be respected in RESTful APIs.

In practice, this means that you can navigate to referenced resources without deeper knowledge of the system. A typical response includes "links" sections for each resource, which can be a sub-resource of a parent resource or other referenced resources. You can use these links to work with the resources.

For example, when you get sales order data, the response includes a customer reference field with a links section. You can then use the link to get data of the particular customer.

For more information, see the following resources:

- <https://en.wikipedia.org/wiki/HATEOAS>
- https://en.wikipedia.org/wiki/Link_relation
- <https://tools.ietf.org/html/rfc5988>

Resource

A resource represents a uniquely identifiable piece of data. Each resource has a unique URL, and can reference other resources.

There are two main types of resources:

- Singular resources
- Collection resources that contain multiple singular resources

Resources can exist in hierarchy, and can form a tree structure, consisting of child and parent resources.

In NetSuite, the most important resource is a record. A record is a singular resource. However, there can be other resources in NetSuite as well.

A record usually references other resources - other records.

An example of a collection resource is a sublist because it contains multiple lines. Each line is a singular child resource, and the record is a parent resource.

A record with multiple sublists, each of them with multiple lines, forms a hierarchical resource.

REST Web Services and Other Integration Options

To choose the best integration option, consider the following comparisons.

The following table compares REST web services, SOAP web services, and RESTlets.

Note: The limits for concurrent requests are unified for SOAP and REST web services and for RESTlets. For information about account concurrency, see the help topic [Web Services and RESTlet Concurrency Governance](#).

Note: For a comparison of RESTlets with other integration options, see the help topic [RESTlets vs. Other NetSuite Integration Options](#).

Attribute	REST Web Services	SOAP Web Services	RESTlets
Supported Operations	get, search, add, update, delete	get, search, add, update, delete	get, search, add, update
Authentication Supported?	Yes (token-based authentication, OAuth 2.0)	Yes (user credentials; token-based authentication)  Important: User credentials are not preferred as an authentication method for SOAP web services. If you're using this method, you should switch to token-based authentication.	Yes (user credentials; token-based authentication, OAuth 2.0)  Important: As of the 2021.1 release, newly created RESTlets don't support user credentials authentication. If you attempt to authenticate a new RESTlet with user credentials after your account is upgraded to 2021.1, an HTTP error response is returned. For more information, see the help topic Using User Credentials for RESTlet Authentication.
Passing of Login Details	in OAuth authorization header	in body (SOAP)	in authorization header
Passing of Parameters	all parameters on URL	all parameters in body (SOAP)	GET parameters on URL
Supported Content Types	JSON, Swagger	text/xml (explicit)	JSON, text/xml (explicit)
Environment	lightweight, no coding and script deployment needed on the server side, suitable for mobile devices	heavy programming and deployment environment (C#, Java) no coding and script deployment needed on the server side	lightweight, suitable for mobile devices, bundleable
Functionality	- Metadata catalog. See Working with Resource Metadata - CRUD. See Working with Records.	- CRUD - Search	All functionality available through SuiteScript: - CRUD - Search - Query

Attribute	REST Web Services	SOAP Web Services	RESTlets
	Record Filtering and Query		 Note: SuiteScript does not provide metadata.
Standards	Similar to the REST APIs of other Oracle products	SOAP protocol	No standards
Required User Knowledge	REST programmer API level knowledge	SOAP programmer API level knowledge	JavaScript programmer SuiteScript level knowledge
Performance	Using REST API, fewer calls may be required to accomplish a business flow. Therefore the overall performance may be better than SOAP and CSV.	SOAP web services require more calls to accomplish a business flow than the REST API.	RESTlets are the fastest integration channel. All actions required for a business flow can be executed within a single call.

SOAP Web Services vs. REST Web Services Operation Mapping

The following table maps SOAP operations to their REST equivalents.

SOAP Operation	REST Operation	Notes
add	POST	
addList	—	List operations are currently not supported in REST web services.
attach / detach	—	The attach / detach operation is currently not supported in REST web services. You can use SuiteScript to define or remove a relationship between two records. For more information, see the help topics record.attach(options) and record.detach(options) .
changeEmail	—	Login through user credentials is not supported in REST web services.
changePassword	—	Login through user credentials is not supported in REST web services.
checkAsyncStatus	GET	You can use the async service to retrieve the status of a single async operation.
delete	DELETE	
deleteList	—	List operations are currently not supported in REST web services.
get	GET	
getAccountGovernanceInfo	governanceLimits	
getAll	GET	

SOAP Operation	REST Operation	Notes
getAsyncResult	—	You can use the async service to retrieve the result of a single async operation.
getBudgetExchangeRate	—	The budget exchange rate record is currently not available in REST web services. You can interact with the budget exchange rate record using SuiteScript. For details, see the help topic Budget Exchange Rate .
getCurrencyRate	GET	You can achieve the equivalent in REST by sending a GET request for a specific currency rate record.
getCustomizationId		In REST web services, you can return data about your customization objects by sending a request to the metadata catalog. For information, see Getting Metadata .
getDataCenterUrls	—	REST web services only use account-specific domains.
getDeleted	—	This functionality is not available in REST web services. You can retrieve information about deleted records by using the Deleted Record search type or a SuiteQL query. For information, see the help topic Searching for Deleted Records .
getIntegrationGovernanceInfo	governanceLimits	
getItemAvailability		You can achieve the equivalent in REST by sending a GET request for a specific record.
getList	GET	You can retrieve a list of records of the same record type in REST.
getPostingTransactionSummary	—	You can search for financial reports and summaries in the UI. For information, see the help topic Financial Statements Overview .
getSavedSearch	—	Saved Search isn't supported in REST web services. REST web services supports SuiteAnalytics Datasets. For more information, see Working with SuiteAnalytics Datasets in REST Web Services .
getSelectValue	—	The getSelectValue operation is currently not supported in REST web services.
getServerTime	serverTime	
initialize / initializeList		In REST web services, you can transform a record from one type into another, using data from an existing records. This is the equivalent of the initialize SOAP operation. For more information, see Transforming Records .
search	GET	
searchMoreWithId		In REST web services, you can use collection paging to move between pages of search results.
update	PATCH	
updateList	—	List operations are currently not supported in REST web services.

SOAP Operation	REST Operation	Notes
updateInviteeStatus	PATCH	You can achieve the equivalent in REST by sending a PATCH request for a specific record.
updateInviteeStatusList	—	List operations are currently not supported in REST web services. You can update calendar events one by one. For information, see the help topic updateInviteeStatus .
upsert	PUT	
upsertList	—	List operations are currently not supported in REST web services.

REST Web Services URL Schema and Account-Specific URLs

To access REST web services, you must use account-specific domains. Account-specific domains are unique to your account because they include your account ID in the domain name. These domains don't change even if your account is moved to a different data center.

When using account-specific domains, dynamic domain discovery is not needed.

The format of an account-specific domain name is <account ID>.<service>.netsuite.com. For example, if your account ID is 123456, your account-specific domain for REST web services is 123456.suitetalk.api.netsuite.com

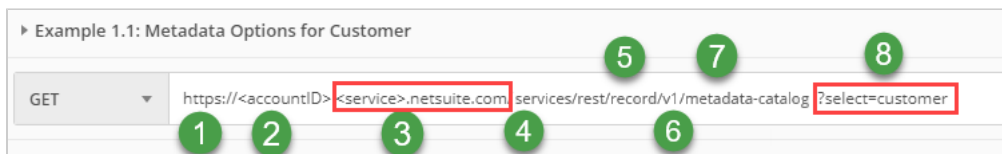
You can find the service URL for SuiteTalk (SOAP and REST web services) at Setup > Company > Setup Tasks > Company Information, on the Company URLs subtab. When you build a client application, make sure the service URL is a configurable parameter.



Important: The examples in this document use demo123 as the account ID. Replace this with your own account ID.

You can access NetSuite resources through REST web services by using URLs specific to either record or query service.

A URL for a REST resource looks like the following.



The URL is made up of following parts:

1. Protocol
2. The account ID of your NetSuite account
3. The domain name for REST web services
4. The complete REST API services endpoint
5. The specific services that is being accessed, for example, record, query, or other service
6. The service version, which is specific to each service

7. The optional sub-service, for example, metadata-catalog, suiteql, or dataset
8. Query parameters

The following examples show the usage of the record and query services.

To use CRUD operations on records, you can use a URL similar to the following:

```
1 | PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/42
```

For information about CRUD operations, see [Working with Records](#).

Metadata-catalog is a subservice of the record service. To access resource metadata, you can use a URL similar to the following:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/?select=customer
```

For information about using metadata, see [Working with Resource Metadata](#).

dataset is a subservice of the query service. To execute saved datasets, you can use a URL similar to the following:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/dataset/ExampleDataset1/result
```

For information about saved datasets, see [Working with SuiteAnalytics Datasets in REST Web Services](#).

Suiteql is a subservice of the query service. To execute SuiteQL queries, you can use a URL similar to the following:

```
1 | POST https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=10&offset=10
```

For information about SuiteQL queries in REST web services, see [Executing SuiteQL Queries Through REST Web Services](#).

Prerequisites and Setup for REST Web Services

To use REST web services, you must enable the relevant features in your account. Also, the REST web services user must have the required permissions assigned to their role. See the following table for details on enabling features and assigning permissions.

Feature or Permission	Usage
REST Web Services feature	By enabling the REST Web Services feature you get access to generally available REST functionality. Enable the feature at Setup > Company > Setup Tasks > Enable Features, in the SuiteTalk (Web Services) section, on the SuiteCloud subtab. To use the feature, you must accept the SuiteCloud Terms of Service.
SuiteAnalytics Workbook feature	Enable the feature at Setup > Company > Setup Tasks > Enable Features, on the Analytics subtab.
Permissions: ■ REST Web Services ■ Log in using Access Tokens ■ SuiteAnalytics Workbook	See Standard Roles with the REST Web Services, SuiteAnalytics Workbook, and Log in Using Access Tokens Permissions for a list of roles that have the required permissions assigned by default. See To assign the required permissions to a user's role for information about assigning the permissions manually.

Note: Using the administrator role for building web services integrations or accessing REST web services isn't recommended for security reasons.

Standard Roles with the REST Web Services, SuiteAnalytics Workbook, and Log in Using Access Tokens Permissions

The following standard roles have the SuiteAnalytics Workbook, Log in using Access Tokens, and REST Web Services permissions assigned by default. For more information about standard roles, see the help topic [NetSuite Roles Overview](#).

- Accountant
- Accountant (Reviewer)
- A/P Clerk
- A/R Clerk
- Bookkeeper
- CEO (Hands Off)
- CEO
- Sales Manager
- Sales Person
- Store Manager

- Support Manager
- Support Person
- Employee Center
- Warehouse Manager
- Payroll Manager
- Partner Center
- Intranet Manager
- Marketing Manager
- Marketing Assistant
- System Administrator
- Sales Administrator
- Support Administrator
- Marketing Administrator
- Advanced Partner Center
- Online Form User
- Engineer
- Issue Administrator
- Payroll Setup
- Engineering Manager
- Product Manager
- CFO
- Retail Clerk (Web Services Only)
- Buyer
- Developer

If you create a new role, or if you use an existing custom role or a standard role that doesn't have the required permissions by default, you can assign the permissions manually.

To assign the required permissions to a user's role:

1. Go to Setup > Users/Roles > User Management > Manage Roles.
2. Find the role you want to modify. Click the corresponding **Edit** or **Customize** link.
3. On the Permissions subtab, click Setup.
4. In the Permission list, select REST Web Services.
5. In the Level list, select Full.
6. Click Add.
7. On the Permissions subtab, click Setup.
8. In the Permission list, select Log in using Access Tokens.
9. In the Level list, select Full.
10. Click Add.
11. On the Permissions subtab, click Reports.
12. In the Permission list, select SuiteAnalytics Workbook.

13. In the Level list, select Edit.
14. Click Add.
15. Click Save.

Setting Up Authentication

NetSuite supports two mechanisms that increase overall system security. OAuth 2.0 and token-based authentication (TBA), robust, industry standard-based mechanisms. These authentication mechanisms enable client applications to use a token to access NetSuite through APIs, eliminating the need for integrations to store user credentials.

REST web services do not support login through user credentials. For general information about TBA, see the help topic [Token-based Authentication \(TBA\)](#).

For general information about OAuth 2.0, see the help topic [OAuth 2.0](#).



Note: OAuth 2.0 is the preferred authentication method. You should consider using OAuth 2.0 instead of TBA whenever possible.

Setting Up OAuth 2.0 Authentication for REST Web Services



Tip: For a step by step description of setting up OAuth 2.0 for REST, see [Tutorial: Using Postman with OAuth 2.0](#).

To set up OAuth 2.0, see the help topic [Getting Started with OAuth 2.0](#).

To be able to use REST web services with OAuth 2.0, you're required to create an application using an integration record. See the help topic [Create Integration Records for Applications to Use OAuth 2.0](#). As the last step of this procedure, make sure you note the client ID and client secret.

After creating the integration application, set up an application for use with OAuth 2.0. For more information, see the help topic [OAuth 2.0 for Integration Application Developers](#). After the application is set up, you can manage the authorized applications. For more information, see the help topic [Managing OAuth 2.0 Authorized Applications](#).

Setting Up Token-Based Authentication (TBA) for REST Web Services



Tip: For a step by step description of setting up TBA for REST, see [Setting Up Token-Based Authentication \(TBA\) - Tutorial](#).

To set up TBA, see the help topic [Getting Started with Token-based Authentication](#).

To use REST web services with TBA, you must create an application using an integration record. See the help topic [Create Integration Records for Applications to Use TBA](#). As the last step of this procedure, make sure you note the consumer key and consumer secret.

After creating the integration application, you're ready to create tokens for your users. Issue a new token for at least one of your users, and note its token ID and token secret. For details, see the help topic [Access Token Management – Create and Assign a TBA Token](#).

After creating tokens, you use the token ID and token secret in your Postman environment. For information about setting up your environment, see [Importing and Setting Up a Postman Environment](#).

For detailed information about using TBA in integration applications, see the help topic [The Three-Step TBA Authorization Flow](#).

REST Web Services Request Processing

Besides synchronous execution, you can also execute requests asynchronously in REST web services.

Note: REST requests have a size limit of 104MB.

The asynchronous execution of requests is useful for long-running requests. With asynchronous requests, you send a request to REST web services, where it is placed in a processing queue and handled asynchronously with other requests. Your client application does not wait for a response. After a job is submitted, a job ID is returned in the REST web services response. Your client application can then check the status and result of the request by referencing the job ID. Asynchronous processing can be useful if you expect your connection to NetSuite to be slow or unstable.

Asynchronous REST jobs are handled within REST Async Processors, according to the number of SuiteCloud Plus licenses. For information about SuiteCloud Plus, see the help topic [SuiteCloud Plus Settings](#).

Note: The processing of asynchronous REST calls does not affect the governance limits of asynchronous SuiteScript processes. The governance limits are calculated independently. However, extensive async REST usage can affect the overall performance so it may cause performance degradation in the running of scripts.

Asynchronous request execution also supports an idempotency retry mechanism, which allows you to handle a particular query through a unique key. Idempotent retry helps to avoid creating duplicate records, and it prevents the submission of the same task multiple times. It also helps you find loose-running jobs after a connection failure.

You can execute any REST request asynchronously by using the `Prefer: respond-async` request header. The response to the request contains the following HTTP header that you can use to retrieve the status of the request: `Location: https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/<id>`.

Sending an Asynchronous Request

You can send a request similar to the following:

```
1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/<id>?fields=companyName,email,entityStatus
2 Prefer: respond-async
```

The following is an example response. The job ID is displayed in the location response header.

```
1 HTTP 202 Accepted
2 Preference-Applied: respond-async
3 Content-Type: application/json
4 Location: /services/rest/async/v1/job/123456
```

Then you can send a request using the job ID in the response header to retrieve the status of the request that is in progress: `GET https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1`.

The following is the response:

```
1 {
2   "links": [
3     {
4       "rel": "self",
```

```

5      "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1"
6    }
7  ],
8  "completed": false,
9  "id": "1",
10 "progress": "pending",
11 "task": {
12   "links": [
13    {
14     "rel": "self",
15     "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task"
16    }
17   ]
18 }
19 }

```

Sending an Asynchronous Request Using an Idempotency Key

The following is an example of a request that contains an idempotency key. The idempotency key enables you to refer to a request through a unique key, and also helps you avoid duplicate requests.

```

1 POST /services/rest/record/v1/account
2 X-NetSuite-idempotency-key: cac827ea-3543-4cda-add2-bae634326c27
3 prefer: respond-async
4 {
5   "name" : "My secret account"
6 }

```

The following is an example of a response you receive after sending a duplicate request.

```

1 HTTP 400 Conflict
2 Preference-Applied: respond-async
3 Location: /services/rest/async/v1/job/123456
4 Content-Type: application/json
5 {
6   "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html#sec10.4.10",
7   "title": "Conflict",
8   "status": 400,
9   "o:errorDetails": [
10    {
11     "detail": "A request with the same idempotency key is already submitted. See the \"Location\" header for a link to the previously submitted job",
12     "o:errorCode": "IDEMPOTENCY_ERROR"
13    }
14   ]
15 }

```

Polling for the Status of a Completed Asynchronous Request

You can send a request similar to the following to get the task ID of the asynchronous request: GET <https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/>

Then you can send a request similar to the following to see the status of a completed asynchronous request: GET <https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1>

The following example shows the returned result:

```

1 {
2   "links": [
3     {
4       "rel": "related",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1/result"
6     },
7     {
8       "rel": "self",
9       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1"
10    },
11  ],
12  "completed": true,
13  "endTime": "2020-12-09T12:04:40Z",
14  "id": "1",
15  "progress": "succeeded",
16  "startTime": "2020-12-09T12:04:28Z"
17 }

```

Getting the Result of an Executed Request

You can send a request similar to the following to get the result of the executed request: GET <https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1/result>.

The following example shows the returned result:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/... "
6     }
7   ],
8   "companyName": "Glenrock General Hospital",
9   "email": "alan.smith@example.com",
10  "entityStatus": {
11    "links": [
12      {
13        "rel": "self",
14        "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customerstatus/13"
15      }
16    ],
17    "id": "13",
18    "refName": "CUSTOMER-Closed Won"
19  }
20 }

```

REST Web Services Operations

The following REST web services operations are available:

- [GovernanceLimits Operation](#)
- [ServerTime Operation](#)

GovernanceLimits Operation

With REST web services, you can use the `governanceLimits` operation to retrieve the available concurrency limit for a specific integration. With this information on limits, the integration can determine how many concurrent requests it can send. Every integration should use this operation to get information about the available limit so as not to exceed the number of available threads that send requests to NetSuite. The `governanceLimits` operation returns results only if you are logged in as an administrator.

In NetSuite governance, managing concurrency limits is critical for performance and resource allocation. The governance information provides metrics on client connections to the system, and governance helps prevent resource contention, while ensuring that integration points operate within defined thresholds.

Request

The `governanceLimits` operation does not have any parameters and returns (1) the account concurrency limit, which specifies the maximum number of concurrent connections allowed for a given account, (2) the unallocated concurrency limit, which indicates a limit for all integrations without a specifically defined limit, and (3) the limit type. There are three limit types.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/system/v1/governanceLimits
```

Responses

There are three responses based on the limit type:

- `integrationSpecific`, when a limit has been allocated to the integration.

```
1 | {
2 |   "accountConcurrencyLimit": 5,
3 |   "accountUnallocatedConcurrencyLimit": 3,
4 |   "integrationConcurrencyLimit": 2,
5 |   "integrationLimitType": "integrationSpecific"
6 | }
```

- `accountLimit`, when no specific limit has been allocated to the integration.

```
1 | {
2 |   "accountConcurrencyLimit": 5,
3 |   "accountUnallocatedConcurrencyLimit": 3,
4 |   "integrationLimitType": "accountLimit"
5 | }
```

- `internal`, when it is an internal application. In this case, the limit is not displayed.

```
1 | {
```

```

2  "accountConcurrencyLimit": 5,
3  "accountUnallocatedConcurrencyLimit": 3,
4  "integrationLimitType": "internal"
5  }

```

ServerTime Operation

The `serverTime` operation returns the NetSuite server time in UTC, regardless of a user's time zone. Developers do not have to rely on client time when writing synchronization procedures because the client time may not be synchronized with the NetSuite server time. For example, if you need to synchronize customer data with NetSuite, you can call `serverTime` to initialize your synchronization process. There are no explicit permissions for this operation.

Note the following possible use cases:

- If you choose, you can write client code that takes the GMT returned by NetSuite and converts it to your local time zone.
- To synchronize new or changed customers records, you can search customers and filter by `lastModifiedDate` using the value returned in `serverTime`. This makes your synchronization process independent of your client time, which may not be synchronized with the NetSuite server time, and hence the timestamp that's used to track changes.

Request

The `serverTime` operation takes no arguments and returns the NetSuite server time, regardless of a user's time zone.

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/system/v1/serverTime

```

Response

The `dateTime` value that is returned by `serverTime` is in UTC time zone. For example:

```

1 | {
2 |   "serverTime": "2025-03-26T16:21:00.000Z"
3 | }

```

Using Postman with REST Web Services

In this document, the REST web services functionality is demonstrated using the [Postman Application](#). However, you can use any similar tool of your preference to work with REST web services. Besides being able to build and send the API requests, the Postman Application also acts as a library for your requests which you can then import and export. You can download Postman at <https://www.getpostman.com/>

Installing Postman

Follow these steps to install Postman:

1. Download and install the Postman desktop application from <https://www.getpostman.com/>.

Note: Do not use the Chrome extension, as it does not support some of the features that are present in the REST web services sample request collection.

2. Run Postman. Click Take me straight to the app on the initial splash screen.
3. Close the initial task window.

Note: If you are using Postman version 10, you must create a user account to be able to send REST requests.

Working with Postman Environments and Collections

A Postman environment is a set of key-value pairs. The key represents the name of the variable. Using a Postman environment, you can switch between various NetSuite accounts, and between your test or production accounts. Using Postman environments, you can customize requests using variables so you can switch between different setups without changing your requests. You can also download environments, save them as JSON files, and upload them later.

A Postman collection is a set of HTTP requests. Similarly to the environments, you can create, share, duplicate, export, and delete a collection. You can also import a collection as a single JSON file. The collection distributed together with this document requires a proper environment setup as described in [Importing and Setting Up a Postman Environment](#).

You can download the REST API Postman environment template and collection of sample requests from the SuiteTalk tools download page at <https://<accountID>.app.netsuite.com/app/external/integration/integrationDownloadPage.nl>. To access the page, you must substitute your account ID in the URL.

Note: To access the Postman environment template and collection, the REST Web Services feature must be enabled, and you must have the REST web services permission assigned to your role. For more information, see [Prerequisites and Setup for REST Web Services](#).

The set of sample requests for the Postman Application is provided to demonstrate how to use the NetSuite REST web services. The sample requests can also help you start building your REST-based integration with NetSuite.

For more information, see the following topics:

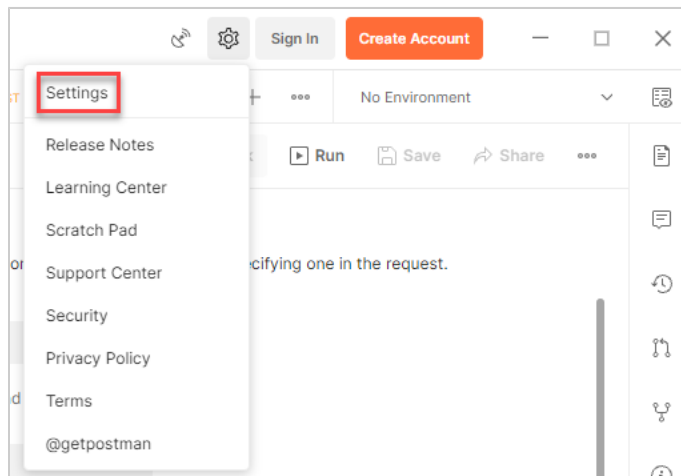
- [Importing and Setting Up a Postman Environment](#)
- [Importing the Postman Collection](#)
- [Sending a Request From the Imported Collection](#)

- [Tutorial: Using Postman with OAuth 2.0](#)
- [Setting Up Token-Based Authentication \(TBA\) - Tutorial](#)

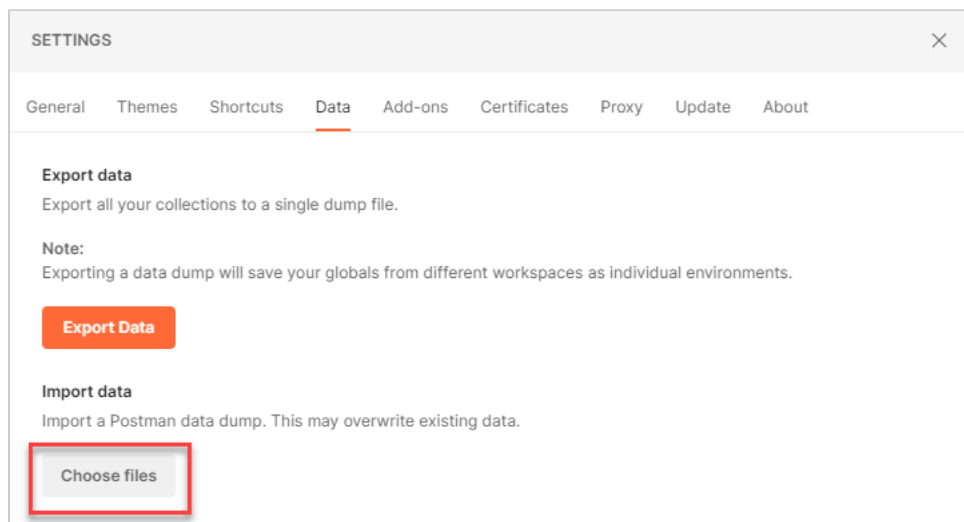
Importing and Setting Up a Postman Environment

To import a Postman collection:

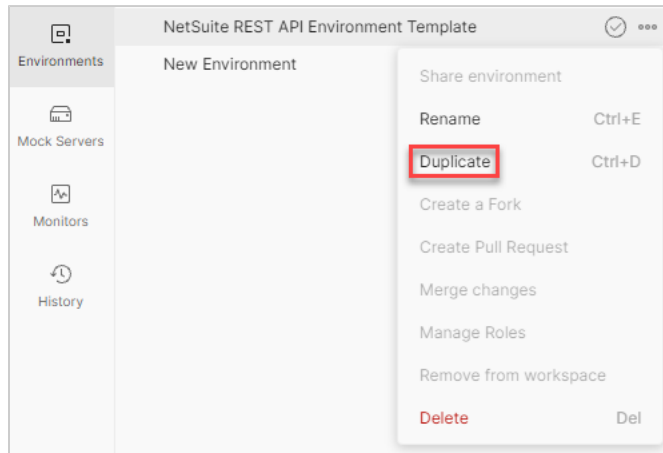
1. Download the Postman environment template and collections archive from the SuiteCloud tools download page at <https://<accountID>.app.netsuite.com/app/external/integration/integrationDownloadPage.nl>. To access the page, you must substitute your account ID in the URL. To access the page, your role must have the REST web services permission assigned to it.
2. Unzip the archive.
3. To import the environment template from the Environment folder, click the gear icon in the top menu of the Postman application, and then click Settings.



4. Go to the Data tab, and then click Choose files to select the template to import. The template is located in the environment folder of the Postman environment template and collections archive.



5. After importing the environment, the REST API Environment Template is displayed in the list of environments. Make a copy of the template for each new TBA token you want to add. Click the Duplicate button to make a copy.



6. Enter a self-descriptive name for your environment, for example, "<role> for <account> at <host>". REST web services only support account-specific domains. For information about account-specific domains, see the help topic [URLs for Account-Specific Domains](#).
7. Enter your account ID (for example, 3604360 or 3604360_SB1 for Sandbox accounts – follow the exact spelling).
8. Enter the TBA credentials you created. For details about creating TBA credentials, see [Setting Up Authentication](#).
9. Enter your company URL for SuiteTalk. You can find your company URL at Setup > Company > Setup Tasks > Company Information, on the Company URLs subtab.

NetSuite REST API Environment Template				
	VARIABLE	TYPE ⓘ	INITIAL VALUE ⓘ	CURRENT VALUE ⓘ
☒	ACCOUNT_ID	default ▾	<ACCOUNT_ID>	<ACCOUNT_ID>
☒	CONSUMER_KEY	default ▾	<CONSUMER_KEY>	<CONSUMER_KEY>
☒	CONSUMER_SECRET	default ▾	<CONSUMER_SECRET>	<CONSUMER_SECRET>
☒	TOKEN_ID	default ▾	<TOKEN_ID>	<TOKEN_ID>
☒	TOKEN_SECRET	default ▾	<TOKEN_SECRET>	<TOKEN_SECRET>
☒	COMPANY_URL	default ▾	<COMPANY_URL>	<COMPANY_URL>
☒	REST_SERVICES	default ▾	{{COMPANY_URL}}/services/rest	{{COMPANY_URL}}/services/rest
	Add a new variable			

Importing the Postman Collection

To import the Postman collection:

1. Download the REST web services Postman collection of sample requests from the SuiteTalk tools download page at <https://<accountID>.app.netsuite.com/app/external/integration/integrationDownloadPage.nl>. To access this page, you must substitute your account ID, and the REST web services permission must be assigned to your role.
2. Unzip the archive.
3. Click Import in the top menu of the Postman Application, and select the NetSuite REST API Tutorial.postman_collection.json file from the requests folder of the downloaded collection.

4. Click the Collections tab in the left panel of the Postman Application to see the newly imported request collection.

Sending a Request From the Imported Collection

Send a test request to verify that your Postman environment configuration and collection import is correct.

To send a test request:

1. Click the label (NetSuite REST API Tutorial) of the newly imported collection. A list of requests is displayed.
2. Open the 0 Test folder in the imported collection, and select Example 0: Test Request from the list.
3. Click Send to run the test request. The response to the request is displayed in the Response section of the Postman window.

If the test request is processed without errors, you see the response **Pass - response should be okay to process** in the Test Results tab of the Postman window.

Tutorial: Using Postman with OAuth 2.0

This tutorial walks you through the setup for OAuth 2.0 in both NetSuite and Postman. Once you're set up, you'll be able to perform CRUD actions on elements of your NetSuite account.

Prerequisites

Before starting, you will need the proper roles, permissions, and features set up in your NetSuite account. For more information, see [Prerequisites and Setup for REST Web Services](#). You will also need to install the Postman desktop application. For more information, see [Installing Postman](#)

Step 1: Integration record for application

1. In your NetSuite account, create a new integration record. Don't save it yet. For how to create this record, see the help topic [Create Integration Records for Applications to Use OAuth 2.0](#).
An integration record is what allows for a connection between Postman and your NetSuite account.
2. In the **Authentication** tab, set up your integration record:

The screenshot shows the 'Authentication' tab in NetSuite. It has two sub-tabs: 'Authentication' (selected) and 'Execution Log'. Under 'Authentication', there are three main sections:

- Token-based Authentication:** Includes checkboxes for 'TOKEN-BASED AUTHENTICATION', 'TBA: ISSUETOKEN ENDPOINT', and 'TBA: AUTHORIZATION FLOW'. There is a text field for 'CALLBACK URL'.
- OAuth 2.0:** This section is expanded. It includes:
 - ☒ AUTHORIZATION CODE GRANT
 - ☐ PUBLIC CLIENT
 - REDIRECT URI:
 - ☐ CLIENT CREDENTIALS (MACHINE TO MACHINE) GRANT
 - SCOPE: ☐ RESTLETS, ☒ REST WEB SERVICES, ☐ SUITEANALYTICS CONNECT
 - APPLICATION LOGO:
 - APPLICATION TERMS OF USE:
 - APPLICATION PRIVACY POLICY:
 - OAUTH 2.0 CONSENT POLICY:
- User Credentials:** Includes a checkbox for 'USER CREDENTIALS'.

- Under **Token-based Authentication**, make sure all options cleared.
 - Under **OAuth 2.0**, make sure only the following options are checked or filled in:
 - **Authorization Code Grant**
 - **REST Web Services**
 - **Redirect URI**: If you're not sure what to enter here, you can use [Postman's callback URL](#).
 - **OAuth 2.0 Consent Policy**: Select whichever option makes the most sense for you. Learn more about [Integration Record and Prompt Parameter Combinations](#).
 - Under **User Credentials**, leave all options cleared.
3. Before clicking **Save**, note that when you do, the client ID and secret will display at the bottom of the page. Make note of both the ID and secret, as you'll need them later and you won't be able to access them again.



Important: The system displays the client ID and client secret only the first time you save the integration record. After you leave this page, these values cannot be retrieved from the system. If you lose or forget the client ID and client secret, you will have to reset them on the Integration page, to obtain new values. Treat these values as you would a password.

4. Click **Save**.

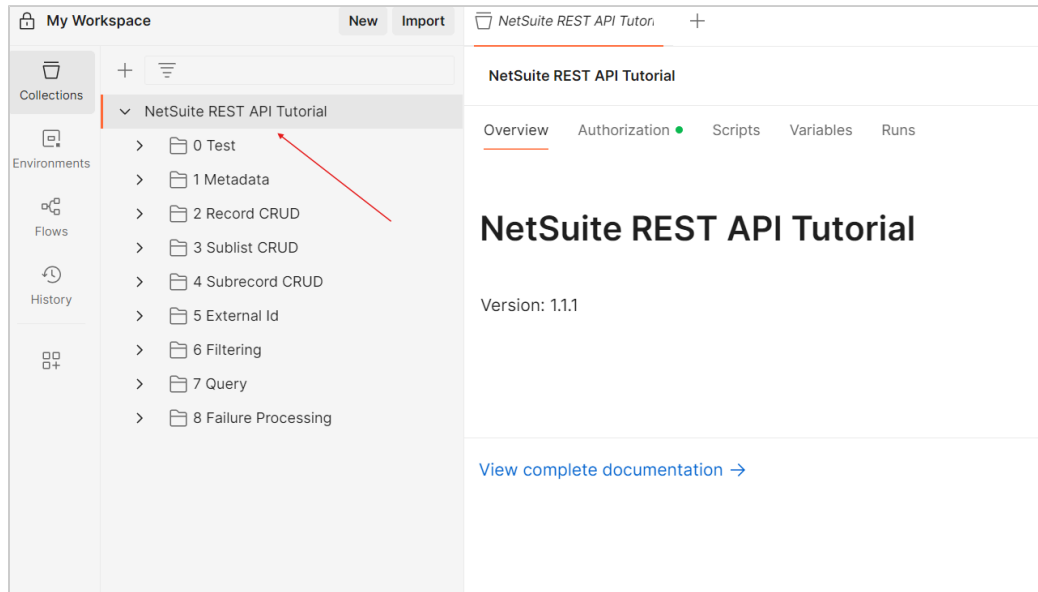
Step 2: Postman environment

1. Follow these steps to import and set up a Postman environment: [Importing and Setting Up a Postman Environment](#).
2. Follow these steps to import a Postman collection: [Importing the Postman Collection](#).
3. Follow these steps to test a request: [Sending a Request From the Imported Collection](#).

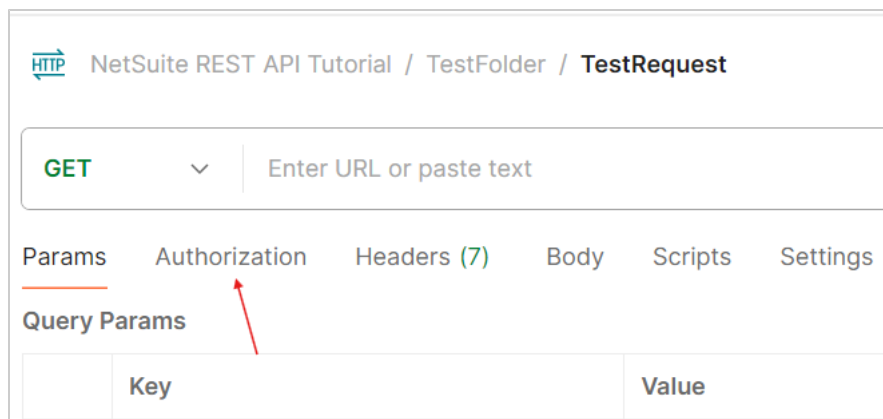
Step 3: Postman token

Creating the token:

1. In Postman, open the **Collections** tab and select your collection that you created in step 2.



2. Open the **Authorization** tab then make the following selections:



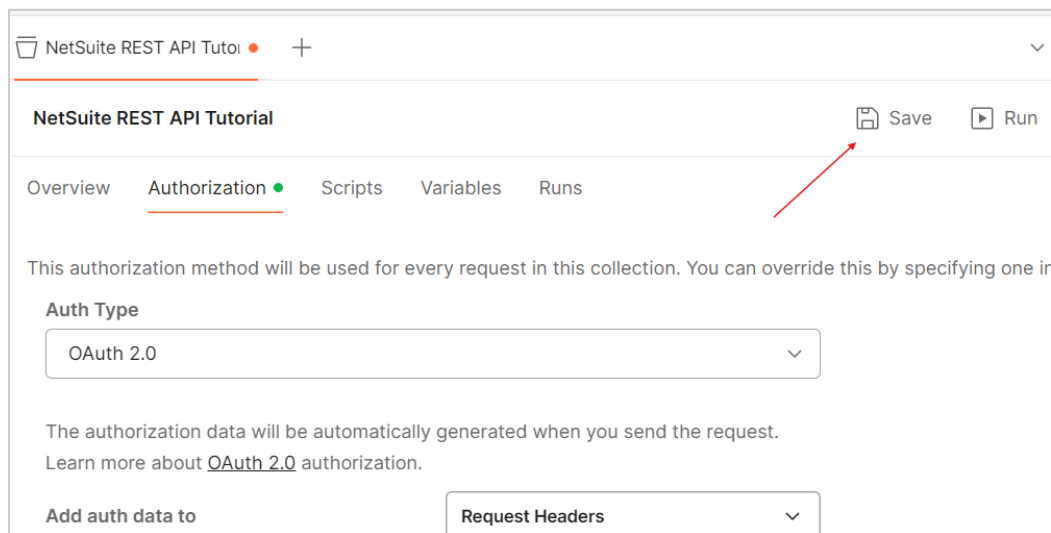
- a. For **Auth Type**, select **OAuth 2.0**.
 - b. For **Add authorization data to**, select **Request Headers**.
3. In the **Configure New Token** section, select or fill in the following:
 - a. **Token name**: Give your token a name.
 - b. **Grant type**: Select **Authorization Code**.
 - c. **Callback URL**: The same URL you used for the Redirect URI.
 - d. **Auth URL**: `https://<youraccountID>.app.netsuite.com/app/login/oauth2/authorize.nl`
Replace **<youraccountID>** with your own account ID.
 - e. **Access Token URL**: `https://<youraccountID>.app.netsuite.com/services/rest/auth/oauth2/v1/token`
Replace **<youraccountID>** with your own account ID.
 - f. **Client ID**: Use the client ID from your integration record.
 - g. **Client Secret**: Use the client secret from your integration record.
 - h. **Scope**: `rest_webservices`

- i. **State:** This can be any random string of ASCII characters. It must include at least 22 characters.
 - j. **Client Authentication: Send as Basic Auth header**
4. Click **Get New Access Token**. If you get the NetSuite login screen after clicking **Get New Access Token**, log in to your account and confirm the message that appears.
 5. Click **Use Token**.



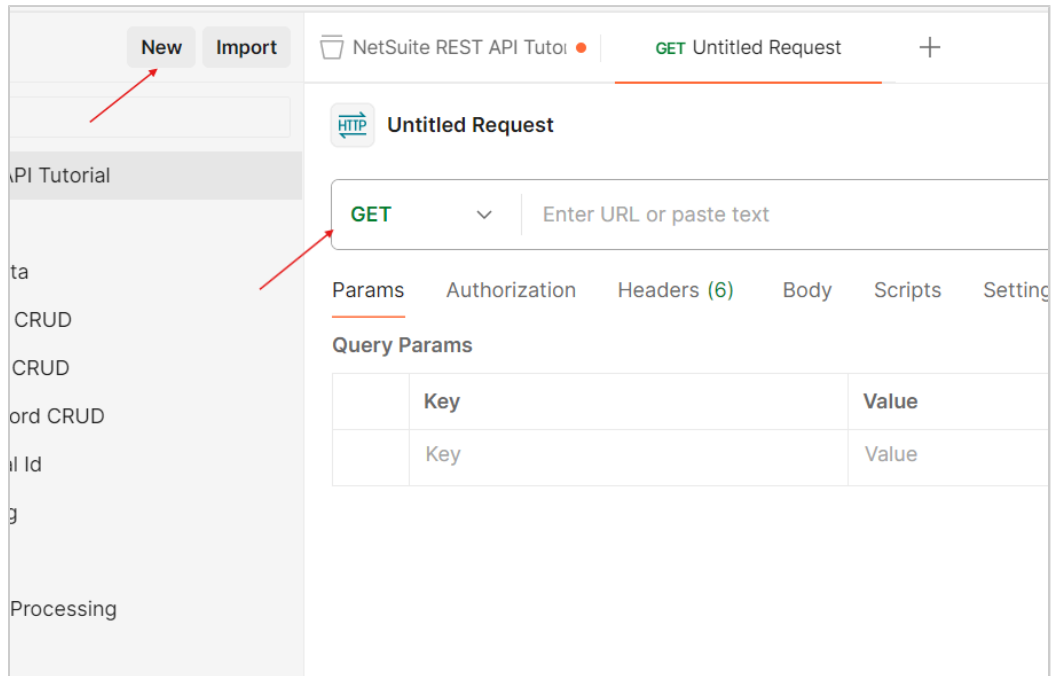
The token will now show up as your selection under **Current Token**.

6. Click **Save**.

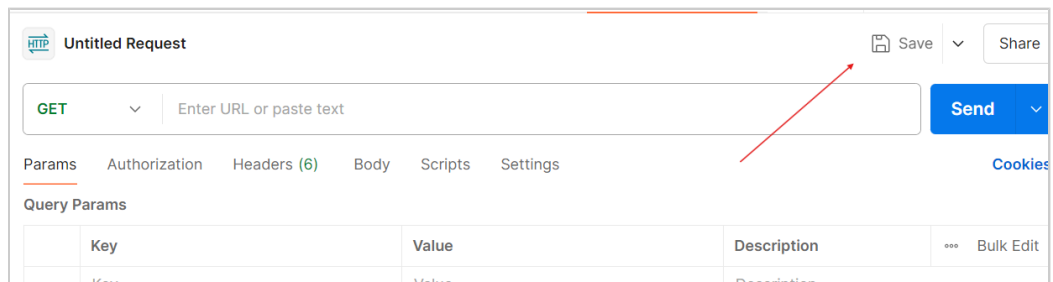


Testing the token:

1. Create a new HTTP GET request.



2. Click **Save**.



3. Select the collection you set up in step 2, then click **New Folder** and create a new folder.

SAVE REQUEST

Request name

Test

[Add description](#)

Save to Select a collection/folder

☰ Search for collection or folder

☰ NetSuite REST API Tutorial

New Collection

Save

Cancel

SAVE REQUEST

Request name

Test

[Add description](#)

Save to **My Workspace / NetSuite REST API Tutorial**

☰ Search for collection or folder

0 Test

1 Metadata

2 Record CRUD

3 Sublist CRUD

4 Subrecord CRUD

5 External Id

6 Filtering

7 Query

New Folder

Save

Cancel

4. Give your request a name and click **Save**.

SAVE REQUEST

Request name

TestRequest

[Add description](#)

Save to **My Workspace / NetSuite REST API Tutorial / TestFolder**

Search for collection or folder

This folder is empty

New Folder

Save Cancel

5. Insert the URL into the GET field. To learn more about this URL, see [REST Web Services URL Schema and Account-Specific URLs](#). For this tutorial, you can use `https://<youraccountID>.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/customer`. Replace `<youraccountID>` with your account ID.

HTTP NetSuite REST API Tutorial / TestFolder / **TestRequest**

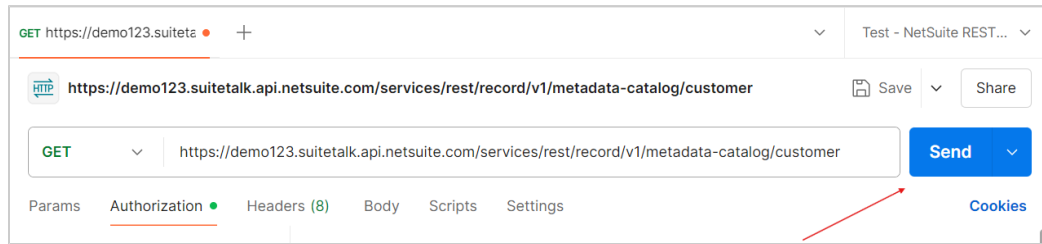
GET Enter URL or paste text

Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value

6. Open the **Authorization** tab.
7. Under **Auth Type**, select **Inherit auth from parent**.
8. Click **Send** to test if the connection works.

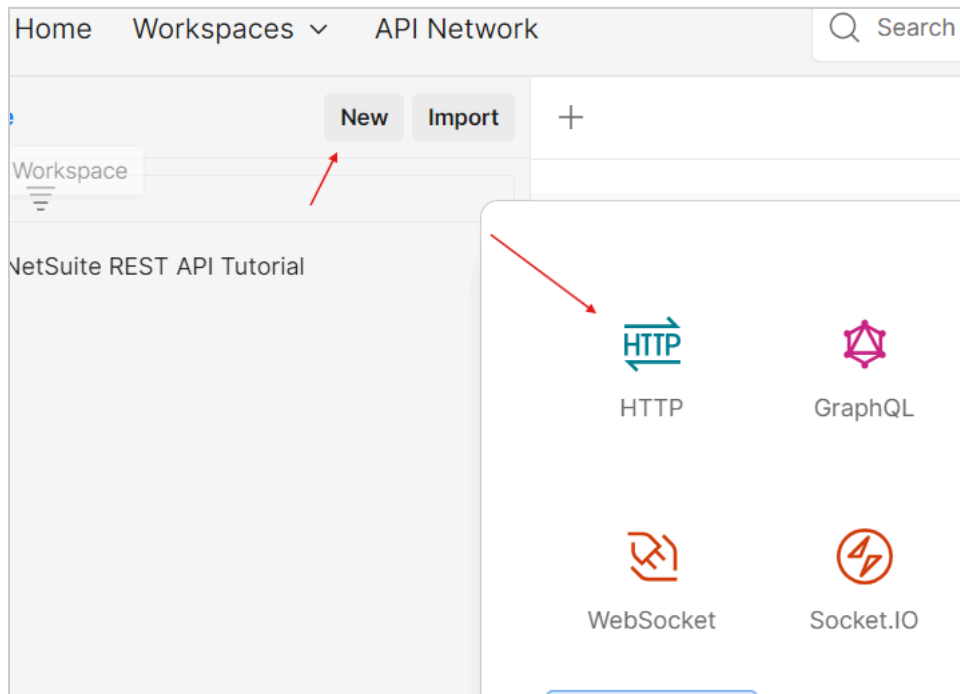


Step 4: CRUD in Postman

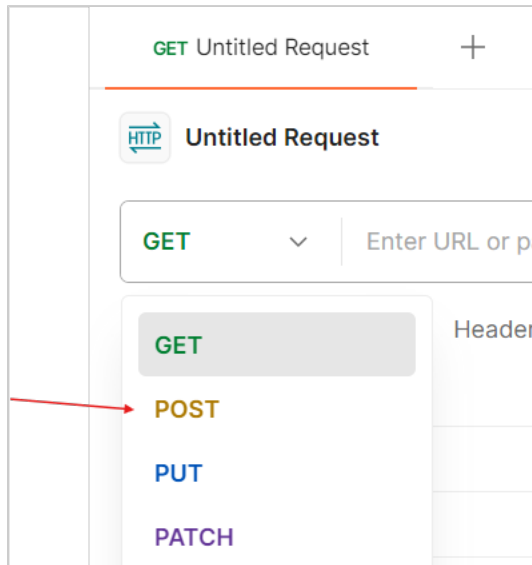
You can now perform CRUD in Postman for your NetSuite account. Try completing the following examples:

Creating a contact:

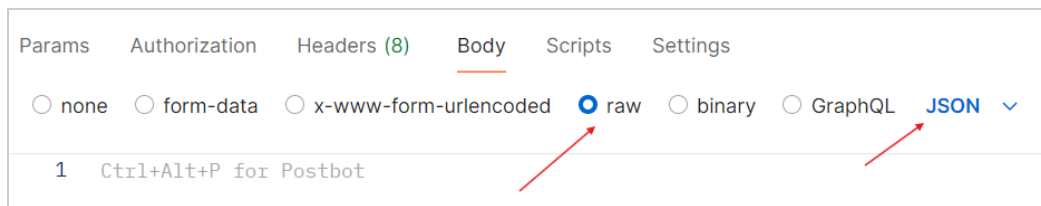
1. Create a new HTTP request.



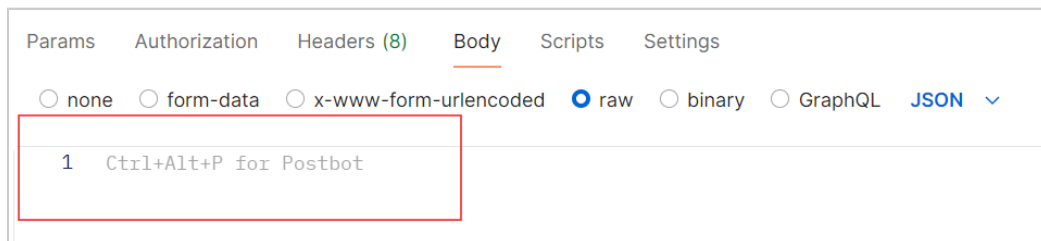
2. Make sure the request is POST.



3. Save this request in the same folder you created when testing the token (see "Testing the token" under "Step 3: Postman token").
4. Under the **Authorization** tab, select **Inherit from parent**.
5. Insert the URL into the POST field. For this example, you can use `https://<youraccountID>.suitetalk.api.netsuite.com/services/rest/record/v1/contact`. Replace `<youraccountID>` with your own.
6. Open the **Body** tab, and select the **raw** option. Make sure the drop-down option has **JSON** selected.



7. Enter the information you want for your contact. At a minimum, you should include an entityId. You can see all the available fields for contact records as well as data types [here](#).



For this example, you can enter the following.

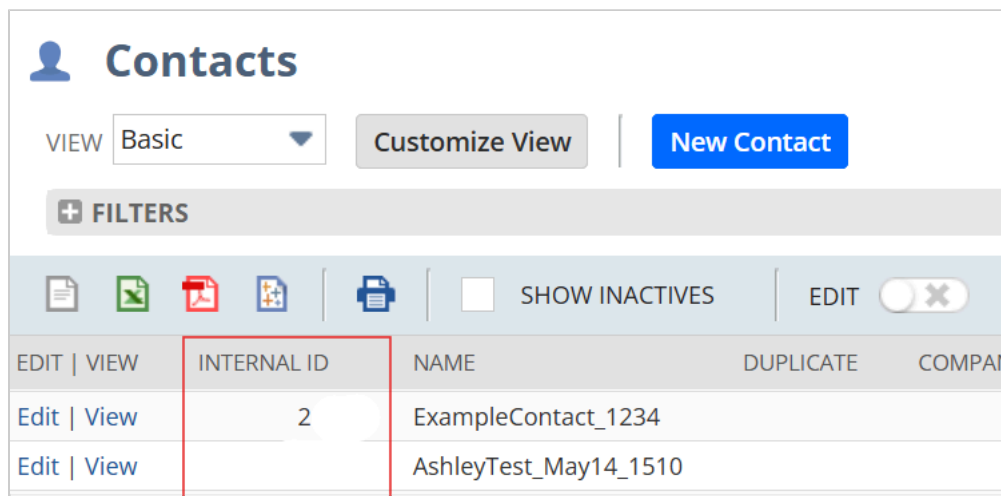
```
1 | {"entityid": "ExampleContact_1234", "comments": "Example text"}
```

8. Click **Send**.
You should see a "204 No Content" message if the POST was successful.

9. If you go into NetSuite, the contact now appears in your contacts list.

Updating a contact:

1. Create a new HTTP request.
2. Make sure the request is a PATCH.
3. Save this request in the same folder you created when testing the token (see "Testing the token" under "Step 3: Postman token").
4. Under the **Authorization** tab, select **Inherit from parent**.
5. Insert the URL into the PATCH field. For this example, you can use `https://<youraccountID>.suitetalk.api.netsuite.com/services/rest/record/v1/contact`
Replace `<youraccountID>` with your own.
6. Add the Internal ID for the contact you just created to the end of the URL. `https://<youraccountID>.suitetalk.api.netsuite.com/services/rest/record/v1/contact/<internalId>`
You can find a contact's internal ID on the Contacts page.



EDIT VIEW		INTERNAL ID	NAME	DUPLICATE	COMPAN
Edit View		2	ExampleContact_1234		
Edit View			AshleyTest_May14_1510		

7. Open the **Body** tab, and select the **raw** option. Make sure the drop-down option has **JSON** selected.
8. Enter the information you want updated for your contact. For this example, you can update the `entityId`. You can see all the available fields for contact and their names as well as data types [here](#).

```
1 | {"entityid": "ExampleContact_5678"}
```

9. Click **Send**.
You should see a "204 No Content" message if the PATCH was successful.
10. Now, if you refresh the page in NetSuite, the contact's name will be updated.

Deleting a contact:

1. Create a new HTTP request.
2. Make sure the request is a DELETE.
3. Save this request in the same folder you created when testing the token (see "Testing the token" under "Step 3: Postman token").
4. Under the **Authorization** tab, select **Inherit from parent**.

5. Insert the URL into the DELETE field. For this example, you can use `https://<youraccountID>.suitetalk.api.netsuite.com/services/rest/record/v1/contact`
Replace `<youraccountID>` with your own.
6. Add the Internal ID to the end of the URL. `https://<your account ID>.suitetalk.api.netsuite.com/services/rest/record/v1/contact/<internalId>`
7. Click **Send**.
You should see a "204 No Content" message if the DELETE was successful.
8. Now, if you refresh the page in NetSuite, the contact will be gone from your contact list.

Setting Up Token-Based Authentication (TBA) - Tutorial

This tutorial walks you through the setup for token-based authentication (TBA) in both NetSuite and Postman. When you're set up, you'll be able to perform CRUD actions on elements of your NetSuite account.



Note: OAuth 2.0 is the preferred authentication method. You should consider using OAuth 2.0 instead of TBA whenever possible. For details, see [Tutorial: Using Postman with OAuth 2.0](#).

Prerequisites

Before starting, you need the proper roles, permissions, and features set up in your NetSuite account. Additionally, you need to install the Postman desktop application. See the following help topics for information about these steps:

- [Prerequisites and Setup for REST Web Services](#)
- [Installing Postman](#)

Creating an Integration Record

After setting up the prerequisites, continue with creating an integration record for your application. For detailed information about integration records, see the help topic [Integration Record Overview](#).

To create an integration record:

1. In your NetSuite account, create a new integration record. Don't save it yet. For information about creating an integration record to be used with TBA, see the help topic [Create Integration Records for Applications to Use TBA](#). An integration record is what allows for a connection between Postman and your NetSuite account.
2. In the **Authentication** tab, set up your integration record.
Under Token-based Authentication, make sure that the Token-based Authentication box is checked. You can leave the rest of the options unchanged.
3. Before clicking Save, note that when you do, the client ID and secret will be displayed at the bottom of the page. Make note of both the ID and secret, as you'll need them later and you won't be able to access them again.

 **Warning:** The system displays the client ID and client secret only the first time you save the integration record. After you leave this page, these values cannot be retrieved from the system. If you lose or forget the client ID and client secret, you will have to reset them on the Integration page, to obtain new values.

Treat these values as you would a password. Never share these credentials with unauthorized individuals and never send them by email.

Creating a TBA Token

After setting up the integration record, you must create and assign a TBA token for the REST web services user.

To create and assign a TBA token:

1. Log in as a user with the **Access Token Management** permission.
2. Go to Setup > Users/Roles > User Management > Access Tokens.
3. On the Access Tokens page, click New Access Token.
The Access token page appears.
4. On the Access Token page:
 - a. Select the **Application Name**.
 - b. Select the **User**.
 - c. Select the **Role**.
 - d. The **Token Name** is already populated by default with a concatenation of Application Name, User, and Role. Enter your own name for this token, if preferred.
 - e. Click **Save**.

The confirmation page displays the Token ID and Token secret.

 **Warning:** For security reasons, the only time the Token ID and Token Secret values are displayed is on the confirmation page. After you leave this page, these values cannot be retrieved from the system. If you lose or forget these credentials, you will need to create a new token and obtain new values.

Treat these values as you would a password. Never share these credentials with unauthorized individuals and never send them by email.

Setting Up TBA in Postman

For setting up TBA in Postman, follow the steps listed in [Importing and Setting Up a Postman Environment](#).

After setting up your authentication, you're ready to start working with REST through Postman. For detailed information, see [Working with Records](#).

Working with Resource Metadata

The REST API consists of a dynamic schema that is described by the metadata catalog. The metadata catalog serves as the API schema defining the contract, similarly to WSDL and XSD files in SOAP web services.

Using the metadata catalog, you can dynamically discover the API, including all available resources, the format and values of input and output, the supported HTTP methods and HTTP query parameters.

In NetSuite, records are the most important resource. The metadata catalog defines all available records, their fields, available values (for example, for enum fields), sublists and subrecords (both standard and custom ones), and their various properties. As new customizations are added to the system, they appear in the metadata catalog as well.

The metadata catalog describes the following properties of the REST API:

- Names of records, fields, sublists, and subrecords
- Field types
- Reference fields and the referenced record types
- Enum fields and their available internal values
- Searchability of records, fields, sublists, and subrecords

The REST API comes with a fully personalized view on the resources (per user). This includes the ability to transparently work with user-specific NetSuite record customizations, such as custom records and fields. The ability to provide a record in its customized form means that the record structure can vary based on your specific NetSuite setup. Therefore, the REST API provides an option to dynamically generate metadata about the records (available resources and operations) in the form of standardized descriptions. Custom record metadata is accessible the same way as metadata for standard records.

Using metadata information, you can:

- Get an overview of all available record types
- Get an overview of the structure of a particular record type
- Get an overview of searchable fields
- Get an overview of supported HTTP methods and query parameters
- Get an overview of keyed and non-keyed sublists.
- Automatically generate client code, for example, API client libraries or client stubs.

REST web services provide metadata in [Swagger \(OpenAPI 3.0\)](#) and [JSON Schema](#) JSON-based formats. Both formats are used to define the structure of JSON data for validation, documentation, and interaction control.

The main difference between the two formats is that the metadata provided in JSON Schema format only describes the internal structure of a resource, for example, its fields, sublists, or subrecords.

In addition to this, the metadata in OpenAPI 3.0 format also describes links to related resources. It also describes how to interact with a resource through REST web services: it describes the URLs, HTTP methods, and parameters.

For more information, see the following help topics:

- [Getting Metadata](#)

- Working with OpenAPI 3.0 Metadata
- Working with JSON Schema Metadata

Getting Metadata

The endpoint for getting the metadata schema for all exposed records is `http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog` where `services/rest` is the name of the REST API endpoint, `record` is the name of the service we are trying to access, `v1` is the service version, and `metadata-catalog` is the sub-resource, that is, the record metadata.



Note: For detailed information about the URL schema for REST web services, see [REST Web Services URL Schema and Account-Specific URLs](#).

The following is an example of a request that returns metadata about all records.

```
1 | GET http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog
```

The following is an example of an intentionally shortened response for the previous request:

```
1 | {
2 |   "items": [
3 |     {
4 |       "name": "account",
5 |       "links": [
6 |         {
7 |           "rel": "canonical",
8 |           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/account",
9 |           "mediaType": "application/json"
10 |        },
11 |        {
12 |           "rel": "alternate",
13 |           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/account",
14 |           "mediaType": "application/swagger+json"
15 |        },
16 |        {
17 |           "rel": "alternate",
18 |           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/account",
19 |           "mediaType": "application/schema+json"
20 |        },
21 |        {
22 |           "rel": "describes",
23 |           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/account"
24 |        }
25 |      ]
26 |    },
27 |  ]
28 | }
```

The response informs you through HATEOAS links about the possible `mediaType` flavor in which the response can be obtained. You can see that the metadata for each record can be served in both OpenAPI 3.0 and JSON schema formats.

You can also create a request that returns the metadata catalog only for selected record types. See the following example.

```
1 | GET http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/?select=assemblybuild,customer,journalEntry
```

✓ **Tip:** Besides getting the full metadata catalog, or the metadata catalog for selected record types, you can also retrieve the metadata catalog in subsequent REST tasks. This option can be useful especially if your account contains a high number of customization objects, such as records or fields.

Working with OpenAPI 3.0 Metadata

To get the metadata in OpenAPI 3.0 format, you have to specify the proper value in the Accept HTTP header. For OpenAPI 3.0, you must add a header in the format `Accept: application/swagger+json`.

To avoid the long loading time needed to gather OpenAPI 3.0 metadata for all records, the metadata catalog resource supports the `select` query parameter for selecting particular record types. You can use this parameter to restrict the metadata to certain record types only. In the following example, the request is modified to restrict the metadata to customer and sales order records only.

```
1 SET-HEADER Accept: application/swagger+json
2 GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog?select=customer,salesorder
```

The following is an intentionally shortened response to the previous request.

```
1  "paths": {
2    "/salesorder": {
3      "get": {
4        "tags": [
5          "salesorder"
6        ],
7        "summary": "Get list of records",
8        "parameters": [
9          {
10           "name": "q",
11           "in": "query",
12           "description": "Search query used to filter results",
13           "schema": {
14             "type": "string"
15           }
16         }
17       ],
18       "responses": {
19         "200": {
20           "description": "Get record",
21           "content": {
22             "application/vnd.oracle.resource+json; type=collection": {
23               "schema": {
24                 "$ref": "#/components/schemas/salesorderCollection"
25               }
26             }
27           }
28         },
29         "default": {
30           "description": "Error response.",
31           "content": {
32             "application/json": {
33               "schema": {
34                 "$ref": "#/components/schemas/rest_error"
35               }
36             }
37           }
38         }
39       }
40     },
41   },
```

The response describes the structure of the records as well as the supported operations, HTTP methods, and query parameters.

The following image shows an excerpt from the response. It describes the record name, the query parameters, and the available HTTP methods you can use with the record.

```

"paths": {
  "/customer/{id}": {
    "get": {
      "tags": [
        "salesorder"
      ],
      "summary": "Get list of records",
      "parameters": [
        {
          "name": "q",
          "in": "query",
          "description": "Search query used to filter results",
          "required": false,
          "schema": {
            "type": "string"
          }
        }
      ],
      "responses": {
        "200": {
          "description": "List of records"
        }
      }
    },
    "post": {
      "tags": [
        "salesorder"
      ],
      "summary": "Create record",
      "parameters": [
        {
          "name": "q",
          "in": "query",
          "description": "Search query used to filter results",
          "required": false,
          "schema": {
            "type": "string"
          }
        }
      ],
      "responses": {
        "200": {
          "description": "Record created"
        }
      }
    }
  },
  "/customer/{id}": {
    "get": {
      "tags": [
        "salesorder"
      ],
      "summary": "Get list of records",
      "parameters": [
        {
          "name": "q",
          "in": "query",
          "description": "Search query used to filter results",
          "required": false,
          "schema": {
            "type": "string"
          }
        }
      ],
      "responses": {
        "200": {
          "description": "List of records"
        }
      }
    },
    "post": {
      "tags": [
        "salesorder"
      ],
      "summary": "Create record",
      "parameters": [
        {
          "name": "q",
          "in": "query",
          "description": "Search query used to filter results",
          "required": false,
          "schema": {
            "type": "string"
          }
        }
      ],
      "responses": {
        "200": {
          "description": "Record created"
        }
      }
    }
  },
  "/salesorder/{id}": {
    "get": {
      "tags": [
        "salesorder"
      ],
      "summary": "Get list of records",
      "parameters": [
        {
          "name": "q",
          "in": "query",
          "description": "Search query used to filter results",
          "required": false,
          "schema": {
            "type": "string"
          }
        }
      ],
      "responses": {
        "200": {
          "description": "List of records"
        }
      }
    },
    "post": {
      "tags": [
        "salesorder"
      ],
      "summary": "Create record",
      "parameters": [
        {
          "name": "q",
          "in": "query",
          "description": "Search query used to filter results",
          "required": false,
          "schema": {
            "type": "string"
          }
        }
      ],
      "responses": {
        "200": {
          "description": "Record created"
        }
      }
    }
  }
},

```

The following excerpt describes the record structure and the properties of different fields: their name, type, format, and attributes.

```

"salesOrder": {
  "type": "object",
  "properties": {
    "defaultILShipMethKey": {
      "type": "integer",
      "format": "int64",
      "nullable": true
    },
    "startDate": {
      "title": "Start Date",
      "type": "string",
      "description": "Sets the date for creating the first invoice. This should only be set if Advanced Billing is being used.",
      "format": "date",
      "nullable": true
    },
    "discountIsTaxable": {
      "type": "boolean"
    },
    "oppcreatedfromtitle": {
      "type": "string",
      "nullable": true
    },
    "clearedDate": {
      "title": "Date Cleared",
      "type": "string",
      "format": "date",
      "nullable": true
    }
  }
},

```

The following excerpt describes a sublist of the sales order record.

```

"salesOrder-giftCertRedemptionElement": {
  "type": "object",
  "properties": {
    "links": {
      "title": "Links",
      "type": "array",
      "readOnly": true,
      "items": {
        "$ref": "#/components/schemas/nsLink"
      }
    },
    "authCodeApplied": {
      "title": "Amount Applied",
      "type": "number",
      "format": "double",
      "nullable": true
    },
    "authCodeAmtRemaining": {
      "title": "Available Credit",
      "type": "number",
      "format": "double",
      "nullable": true
    },
    "giftCertAvailable": {
      "type": "number",
      "format": "double",
      "nullable": true
    },
    "parentTransaction": {
      "$ref": "#/components/schemas/salesOrder"
    },
    "authCode": {
      "$ref": "#/components/schemas/nsResource"
    }
  }
},

```

The OpenAPI 3.0 record metadata received in the previous step can be used as input for any OpenAPI 3.0 compatible tool for further processing. For example, you can use the metadata in tools that generate REST client stubs as well as tools that specialize in dumping a static webpage that describes record structure. In this example, OpenAPI 3.0 metadata is used in the freely available online [Swagger Editor](#) for exploring the structure and operations that can be done with the customer and sales order records.

Note: The performance of the online Swagger editor can be limited if there is a large amount of data to process. Therefore you shouldn't use the editor to generate metadata description for all exposed records.

By copy-pasting the example response to the Swagger Editor, you can see an output similar to the following:

SalesOrder (Beta)			▼
GET	/salesOrder	Get list of records.	🔒
POST	/salesOrder	Insert record.	🔒
GET	/salesOrder/{id}	Get record.	🔒
PUT	/salesOrder/{id}	Insert or update record.	🔒
DELETE	/salesOrder/{id}	Remove record.	🔒
PATCH	/salesOrder/{id}	Update record.	🔒
POST	/salesOrder/{id}/!transform/fulfillmentRequest	Transform to fulfillmentRequest.	🔒
POST	/salesOrder/{id}/!transform/itemFulfillment	Transform to itemFulfillment.	🔒
POST	/salesOrder/{id}/!transform/returnAuthorization	Transform to returnAuthorization.	🔒
POST	/salesOrder/{id}/!transform/cashSale	Transform to cashSale.	🔒
POST	/salesOrder/{id}/!transform/revenueCommitment	Transform to revenueCommitment.	🔒
POST	/salesOrder/{id}/!transform/invoice	Transform to invoice.	🔒
POST	/salesOrder/{id}/!transform/storePickupFulfillment	Transform to storePickupFulfillment.	🔒

Note: Links to NetSuite records that are out of the scope of the select parameter are declared as generic JSON objects with no structure in the returned record metadata. You can avoid this by listing the referenced record in the select parameter.

From the graphic representation, you can see how to obtain a list of all customer and sales order records, how to create new record instances, and how to perform read, update, and delete operations upon them. You can expand each REST method for more description. The Swagger Editor output also contains a **Schema** section. The figure below shows how to use it to explore the structure of a record. For instance, you can see that the salesorder record contains the item sublist, represented in the form of salesorder-itemCollection that contains the totalResults, links, and items properties. The structure of a single line of the item sublist is then captured in the salesorder-itemElement part.

salesOrder-itemCollection ▾ {	
description:	This record is available as a beta record.
links	Links ▾ {
	title: Links
	readOnly: true
totalResults	nsLink > {...}]
	integer(\$int64)
	title: Total Results
	readOnly: true
count	integer(\$int64)
	title: Count
	readOnly: true
hasMore	boolean
	title: Has More Results
	readOnly: true
offset	integer(\$int64)
	title: Query Offset
	readOnly: true
items	▾ [salesOrder-itemElement ▾ {
	description:
	This record is available as a beta record.
	links
	Links > [...]
	rateSchedule
	string
	nullable: true
	string(\$date)
	title: Item Options Date
	nullable: true
	x-ns-custom-field: true
	quantityFulfilled
	number(\$float)
	title: <NULL>
	nullable: true
	oldItemId
	string
	nullable: true
	altId
	integer(\$int64)
	nullable: true
	shipCarrier
	string
	title: Carrier
	nullable: true
	Enum:
	> Array [2]
	refAmt
	number(\$double)
	nullable: true
	revrecterminmonths
	integer(\$int64)
	nullable: true
	commitInventory
	string
	title: Commit
	nullable: true
	Enum:
	> Array [3]

Working with JSON Schema Metadata

Similarly to OpenAPI 3.0 metadata, you can obtain the JSON Schema description by setting the value of the Accept HTTP header to `application/schema+json`, as in the following example.

```
1 | SET-HEADER Accept: application/schema+json
2 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/customer
```

The following is a shortened example of a response to the previous request. Note that the metadata provided in JSON Schema format only describes the internal structure of a resource, for example, its fields, sublists, or subrecords, and links to related resources, and does not describe URLs, HTTP methods, and parameters.

```

1 {
2   "type": "object",
3   "properties": {
4     "custentity_integer": {
5       "title": "Integer Number",
6       "type": "integer",
7       "format": "int64",
8       "nullable": true,
9       "x-ns-custom-field": true
10    },
11    "startDate": {
12      "title": "Start Date",
13      "type": "string",
14      "description": "<p>Enter the date this person or company became a customer, lead or prospect.</p><p>If this person or
company has a contract with you, enter the start date of the contract.</p><p>If you enter an estimate or an opportunity for this
customer, this field will be updated with the date of that transaction.</p>",
15      "format": "date",
16      "nullable": true
17    },
18  }
19 },
20
21 "$schema": "https://json-schema.org/draft-06/hyper-schema#"

```

Note: The JSON generated by the REST API generates external URL links leading to metadata about the linked record types. These are, however, reachable only locally and are unreachable for external validators (such as the JSON Schema validator).

The JSON Schema record metadata received in the previous step can be used as input for any JSON Schema Draft 6 compatible library and tool for further processing. The most frequent use of the JSON Schema record metadata is input and output validation. The following JSON object is an example representing a customer record.

```

1 {
2   entityid: "My Customer",
3   currency: { "id": "1" },
4   representingsubsidiary: { "id": "1" },
5   monthlyclosing: "31 - End of the Month"
6 }

```

You can check if this record representation conforms to the record metadata schema by using the freely available [JSON Schema Validator](#).

 **JSON Schema Validator** newtonsoft.com

An online, interactive JSON Schema validator. Supports JSON Schema Draft 3, Draft 4, Draft 6 and Draft 7. [View source code](#)

Select schema:

1 {
2 "required": [
3 "currency",
4 "entityid",
5 "monthlyclosing"
6],
7 "type": "object",
8 "properties": {
9 "links": {
10 "title": "links",
11 "type": "array",
12 "readOnly": true,
13 "items": {
14 "type": "object",
15 }
16 },
17 "parent": {
18 "title": "Parent",
19 "type": "object",
20 },
21 "referrerlist": {
22 "title": "Referrer",
23 "type": "object",
24 },
25 "startdate": {
26 "title": "Start Date",
27 "type": "string",
28 "format": "date"
29 }
30 }

1 {
2 entityid: "My Customer",
3 currency: { "id": "1" },
4 monthlyclosing: "31 - End of the Month"
5 }

✓ No errors found. JSON validates against the schema

SuiteTalk REST Web Services

ORACLE NetSuite

Working with Records

Using REST Web Services for Custom Record Operations

Using REST web services, you can perform CRUD (create, read, update, delete) operations on NetSuite records. The following sections provide information about the structure of NetSuite records, and the ways you can work with records using REST web services.

 **Note:** You can use APM to monitor the performance of your REST web services integration. For information, see the help topic [Using REST Web Services Analysis](#).

- [The REST API Browser](#)
- [NetSuite Record Structure](#)
- [Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1](#)
- [Creating a Record Instance](#)
- [Getting a Record Instance](#)
- [Updating a Record Instance](#)
- [Using the Upsert Operation](#)
- [Deleting a Record Instance](#)
- [Accessing Subresources in REST Web Services](#)
- [Using External IDs](#)
- [Using Datetime Fields](#)
- [Executing Record Actions](#)
- [Transforming Records](#)
- [Working with the Pricing Sublist on Item Records](#)
- [Using the REST Web Services SuiteScript Execution Context](#)

The REST API Browser

[Go to the REST API Browser.](#)

The REST API Browser provides a visual overview of the structure and capabilities of the REST web services Record API. The data presented in the REST API Browser is based on OpenAPI 3.0 metadata. For more information about metadata, see [Working with OpenAPI 3.0 Metadata](#).

The REST API browser provides the following information:

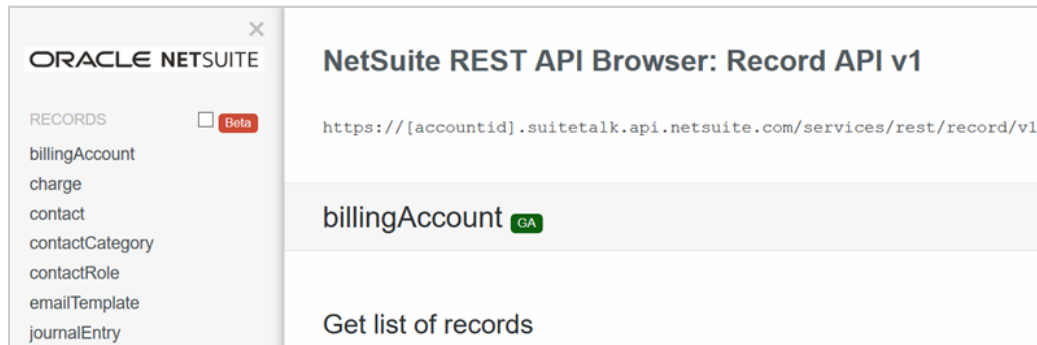
- The summary of all records, sublists, schema definitions, and other objects.
- The available operations you can perform on a record.
- The description of URLs, HTTP methods, and request parameters used for CRUD operations.
- The structure of responses you can receive after performing an operation.
- The structure of error messages.
- The description of field names and field types, and the properties of fields.
- The subformat of strings associated with specialized fields such as date and time fields.

Navigating in the REST API Browser

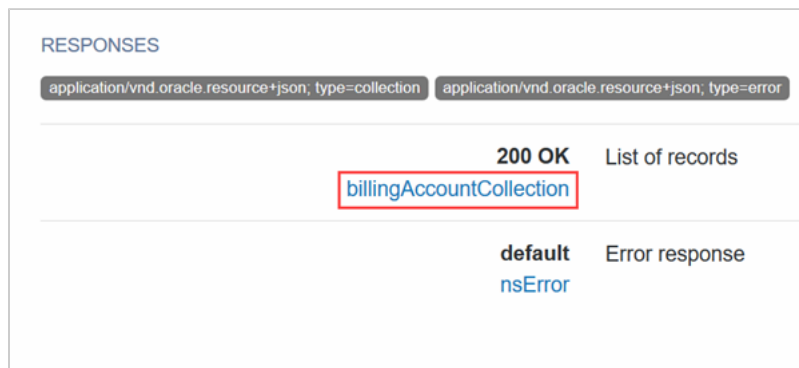
The REST API Browser is designed to let you browse for records and schema definitions in an alphabetical order.

To view records and schema definitions:

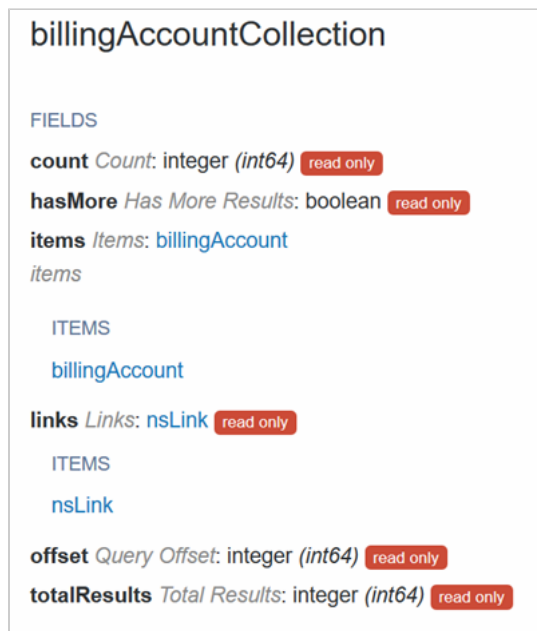
1. Click the name of the appropriate record in the left pane.



2. Review the path, the request parameters, and the responses.
3. Click the linked resource in the Responses section. By clicking the link, you can navigate to the schema definitions section of the REST API Browser.



4. In the schema definitions section, review the properties of the resource.



NetSuite Record Structure

The figure below shows a standard customer record in NetSuite.

Customer Alan Rath

Buttons: Edit, Back, Accept Payment, Auto Close Items, Actions

Primary Information

Customer Id	Sales Rep	Default Order Priority
Alan Rath	Mathew Christner	
Type	Partner	Default Allocation Strategy
Company		Comments
Company Name	Web Address	
Glenrock General Hospital		
Status	Category	
CUSTOMER-Closed Won		

Tabs: Relationships, Communication, **Address**, Sales, Support, Access, System Information, E-Document, SuiteCommerce Extensions, Outbound SSO, Fixed Assets, Geography

ID	Default Shipping	Default Billing	Residential Address	Label	Address
236	Yes	Yes		Bill to	Alan Rath Glenrock General Hospital 417 Washington Blvd Glenrock WY 82637

The figure outlines the basic components of the record:

1. Body fields - for example, name, ID, or email. Body fields are placed either on the main area of the record or on a subtab.
2. Sublists - for example, the Info sublist on a customer record. A sublist consists of line items and their fields.
3. Subrecords - for example, the address subrecord. A subrecord includes many of the same elements as standard NetSuite records, for example, body fields, sublists, and sublist fields. However, you can only create, edit, remove, or view a subrecord from within the context of its parent record.

You can use REST web services to get and set values on body fields and sublists. You can also use REST web services to create these components.

Creating, updating, and deleting subrecords is not supported.

Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1

REST web services enables you to perform CRUD (create, read, update, and delete) operations on custom record instances. You can use CRUD operations to perform business processing on custom records.

In NetSuite, you track all of the information in your account using records. Users with the Custom Record Entries permission can create custom records that collect information specific to the needs of their business.

You can also use all CRUD operations on custom list records and on basic, journal, and header only custom transactions. Sales and purchase custom transaction records are not supported. For information about custom transaction styles, see the help topic [Custom Transaction Styles Overview](#).

You can use CRUD operations to perform business processing on custom lists and custom transactions.

For information about working with custom lists in the UI, see the help topic [Custom Lists](#). For information about working with custom transactions in the UI, see the help topic [Custom Transactions](#).

Usage Notes for Custom Records

Consider the following information when you perform CRUD operations on custom records.

- Creating custom record types through REST web services is not supported. To work with custom record definitions, you must have the Custom Records feature enabled on your account. To enable the feature, go to Setup > Company > Setup Tasks > Enable Features, and select Custom Records on the SuiteCloud subtab. You must create a custom record type in the UI, and then you can work with instances of the custom record type through REST web services. For information, see the help topic [Creating Custom Record Types](#). You also can use SDF to create custom record types. For information, see the help topic [Custom Record Types as XML Definitions](#).
- The record owner of a custom record can be changed through REST web services even if the Allow change option is not selected on the custom record. This behavior is the same as in SuiteScript.
- The script ID field is always displayed through REST web services, even if the Show ID field is not set in the UI. This is standard behavior in non-UI channels for accessing NetSuite to make sure that the records can be identified.
- When working with custom records, you must refer to the custom record in your request by using its unique script ID. For information about the name and ID settings of custom record types, see the help topic [Entering Name and Display Settings](#).

Custom Record Type
REST_custom_record

Save Cancel Change ID Actions

Name *
REST_custom_record

ID
customrecord783

Internal ID
783

- In parent-child record relationships, REST web services do not display children on the parent record. This behavior is different from the behavior of SuiteScript. You can use record collection filtering to access only child records that belong under a particular parent.

When working with custom records, you follow the same procedures as when you work with standard records. You use CRUD operations or get the metadata of custom records in the same way as you do for standard records. For information about using CRUD operations and getting metadata, see the following help topics.

- [Creating a Record Instance](#)
- [Getting a Record Instance](#)
- [Updating a Record Instance](#)
- [Deleting a Record Instance](#)
- [Working with Resource Metadata](#)

Creating a Record Instance

In REST web services, you can create a new record using the POST HTTP method. The POST method expects a request body (a JSON object) that conforms to the record's metadata schema and contains values for at least each mandatory field of the given record type.

REST web services do not support the update of subrecords, and do not support the bulk creation of records.

Note: Not all record fields can be set using the POST method. For example, the id field is read-only.

Any omitted fields are considered empty or to have default values. The following example shows how to create a new customer record instance.

```
1 | POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
2 | { "entityid": "New Customer", "companyname": "My Company", "subsidiary": { "id": "1" } }
```

The following is an excerpt from the response headers of a successful operation, with no body content returned (HTTP Code 204). The URL of the newly created record is given in the Location header of the response.

```
1 | Date Fri, 04 Jan 2019 08:50:20 GMT
2 | Location https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/647
3 | Content-Type application/json
```

Getting a Record Instance

In the following example, an instance of the customer record type created in the previous section is retrieved. You can get a particular customer instance by sending a request in the following form: /services/rest/record/v1/customer/<id>. In the following example, the id of the newly created record is 107. You can read the record instance using the following request.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107
```

The following is an excerpt from the response headers of a successful operation, returned with HTTP Code 200.

```
1 | Date Thu, 14 Feb 2019 14:05:27 GMT
2 | Content-Length 6246
3 | Content-Type application/vnd.oracle.resource+json; type=singular
```

The following is a shortened example of the body of the response.

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
6 |     }
7 |   ],
8 |   "accessRole": {
9 |     "links": [
10 |      {
11 |        "rel": "self",
12 |        "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/"
13 |      }
14 |    ],
15 |     "id": "14",
16 |     "refName": "Customer Center"
17 |   },
18 |   "accessTabChanged": false,
```

```

19     "addressbook": {
20       "links": [
21         {
22           "rel": "self",
23           "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/addressbook"
24         }
25       ]
26     },
27     "autoName": true,
28     "balance": 0,
29     "billPay": false,
30     "bulkmerge": {
31       "links": [
32         {
33           "rel": "self",
34           "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/bulkmerge"
35         }
36       ]
37     },

```

In the following example, an instance of a custom record type is retrieved. You can get a particular custom record type instance by sending a request in the following format, using the script ID of the custom record type: `/services/rest/record/v1/customrecord_api_rest/<id>`. In the following example, the internal ID of the custom record type instance is 123. You can read the record instance using the following request.

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customrecord_api_rest/123

```

The response is in a format that conforms to the Swagger and JSON schema. You can validate the retrieved data against the JSON Schema. For information about validation, see [Working with JSON Schema Metadata](#).

In the response, note the Hypermedia As The Engine Of Application State (HATEOAS) links elements. Using the links elements, you can navigate through the REST endpoint.

Getting Body Fields on a Record Instance

You can also retrieve specific body fields of a record by using the GET method.

You can use a request similar to the following URL with parameter fields to specify the list of fields you want to retrieve:

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107?fields=companyName,email,entityId

```

The response contains only the fields you specify in the request.

```

1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107?fields=companyName%2Cemail%2CentityId"
6 |     }
7 |   ],
8 |   "companyName": "Glenrock General Hospital",
9 |   "email": "alan.rath@ns.com",
10 |  "entityId": "Alan Rath"
11 | }

```

You can selectively retrieve body fields from a record instance, but the selective retrieval of sublist and subrecord fields is not supported.

Format of Sublists and Subrecords

REST web services do not automatically expand sublists and subrecords. You can use the `expandSubResources` query parameter to expand sublists and subrecords. If the query parameter is not used, the response contains only the body fields of the record, and the sublists and subrecords are represented by links. See the example in [Getting a Record Instance](#).

The following is an example where the query parameter is used.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107?expandSubResources=true
```

In the response, note the expansion of the `addressbookaddress` subrecord.

```
1 | "accessTabChanged": false,
2 |   "addressbook": {
3 |     "links": [
4 |       {
5 |         "rel": "self",
6 |         "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/addressbook"
7 |       }
8 |     ],
9 |     "items": [
10 |      {
11 |        "links": [
12 |          {
13 |            "rel": "self",
14 |            "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/addressbook/39"
15 |          }
16 |        ],
17 |        "addressbookaddress": {
18 |          "links": [
19 |            {
20 |              "rel": "self",
21 |              "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/address
22 | book/39/addressbookaddress"
23 |            }
24 |          ],
25 |          "addr1": "417 Washington Blvd",
26 |          "addressee": "Glenrock General Hospital",
27 |          "addrText": "Alan Rath\nGlenrock General Hospital\n417 Washington Blvd\nGlenrock WY 82637",
28 |          "attention": "Alan Rath",
29 |          "city": "Glenrock",
30 |          "country": "US",
31 |          "dropdownstate": {
32 |            "links": [
33 |              {
34 |                "rel": "self",
35 |                "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/"
36 |              }
37 |            ],
38 |            "id": "WY",
39 |            "refName": "Wyoming"
40 |          },
41 |          "id": "152",
42 |          "nKey": "152.0",
```

Format of Selects and References

References to records in REST web services contain an internal ID (`id`), a reference name (`refName`) (that is, the value obtained by invoking the `getFieldText()` method on the reference field), and an HATEOAS link navigating to the referenced record.

```
1 | "currency": {
```

```

2 |     "links": [
3 |         {
4 |             "rel": "self",
5 |             "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/1"
6 |         }
7 |     ],
8 |     "refName": "USD",
9 |     "id": "1"
10 | }

```

Format of Multiselects

REST web services models multiselects as a collection of references containing the internal ID and HATEOAS links navigating to the referenced record. The following example shows the subsidiary multiselect field that is located on the account record.

```

1 | "subsidiary": {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account/106/subsidiary"
6 |     }
7 |   ],
8 |   "items": [
9 |     {
10 |      "links": [
11 |        {
12 |          "rel": "self",
13 |          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subsidiary/1"
14 |        }
15 |      ],
16 |      "id": "1"
17 |    },
18 |    {
19 |      "links": [
20 |        {
21 |          "rel": "self",
22 |          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subsidiary/2"
23 |        }
24 |      ],
25 |      "id": "2"
26 |    }
27 |   ],
28 |   "totalResults": 2
29 | }
30 |

```

Format of Enumeration Values

Prior to 2021.1, the values of enumerator fields were returned in an enumeration-key format, and the internal ID of the enumerator field was used in the REST response. As of 2021.1, both the internal ID and the UI label of enumerator fields are returned by default. You can use the `simpleEnumFormat` query parameter to control the format in which enumerator field values are returned. If the `simpleEnumFormat` parameter is not used or it is set to `false`, enumerator field values are returned in the new format. If the `simpleEnumFormat` parameter is set to `true`, only the internal ID of the enumerator field is returned.

The following example shows a response returned in the new format.

```

1 | GET /record/v1/contact/81
2 | {
3 |   "globalSubscriptionStatus": {
4 |     "id": "2",

```

```

5  "refName": "Soft Opt-Out"
6  }
7  }

```

The following example shows a request where the `simpleEnumFormat` query parameter is set to `true` to specify that the response should be returned in the response format that was used prior to 2021.1.

```

1  GET /record/v1/contact/81?simpleEnumFormat=true
2  {
3    ...
4
5    "globalSubscriptionStatus": "2"
6
7    ...
8  }

```

Additionally, as of 2021.1, when you use the POST, PATCH, and PUT operations, you can specify enumerator field values in the REST request body in both formats, by using internal IDs and reference names. Prior to 2021.1, the field values could only be changed by using internal IDs. You should start using the new format, and use reference names in POST, PATCH and PUT requests.

The following example shows a POST request using the new format.

```

1  POST /record/v1/contact/81
2
3  {
4    "globalSubscriptionStatus": {
5      "refName": "Soft Opt-Out"
6    }
7  }

```

Updating a Record Instance

In the following example, the name of a specific customer record instance is updated. In REST web services, you can perform such an update using the PATCH HTTP method. The PATCH method expects a request body with the same fields that can be retrieved using the GET method. That is, the record's metadata schema is shared between reads and updates.

Note: Not all record fields can be updated using the PATCH method. For example, the `id` field is considered to be read-only.

Any omitted fields are considered unchanged.

In this example, the name (`entityid`) of the record instance 107 retrieved in the previous example is changed from "Alan Rath" to "Updated Customer". Send the following request to perform the update.

```

1  PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107
2  BODY { "entityid": "Updated Customer" }

```

The following is an excerpt from the response headers of a successful operation, with no body content returned (HTTP Code 204). The URL of the updated record is given in the Location header of the response.

```

1  Date Fri, 04 Jan 2019 09:07:01 GMT
2  Location https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107
3  Content-Type application/json

```

Deleting Body Fields from a Record

To delete body fields from a record, consider the following. If the value of a body field is set to null, the body field is deleted. You can also use the `replaceSelectedFields=true` query parameter to indicate that only the body fields included in the query parameter with the value set to null should be deleted.

```
1 PATCH ../myrecord?replaceSelectedFields=true&replace=body1
2
3 {
4   "body1" : null
5 }
```

Using the Upsert Operation

The upsert operation enables you to either create a record, or update an existing record. You can only use the upsert operation when you use an external ID in the request URL and when you use the PUT HTTP method. For information about using external IDs, see [Using External IDs](#).

You can use the upsert operation as a synchronization tool. When using the upsert operation, you do not need to be concerned whether the record with the given external ID already exists.

The following example shows how to use the upsert operation. If the record does not exist, it will be added. If the record already exists, it will be updated.

```
1 PUT http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/eid:CID002
2 {
3   "firstName": "John",
4   "lastName": "Smith"
5 }
```

Deleting a Record Instance

To delete a record instance, you need to specify the record type and the instance identifier. The following is a delete request.

```
1 DELETE https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107
```

The following is an excerpt from the response headers of a successful operation, with no body content returned (HTTP Code 204).

```
1 Date Thu, 14 Feb 2019 15:38:21 GMT
2 Content-Type application/json
```

Accessing Subresources in REST Web Services

In NetSuite, the most important resource is a record. A record is a singular resource. However, there can be other resources in NetSuite as well.

A record usually references other resources - other records. For more information about resources, see [Resource](#).

Sublists and subrecords are subresources. Subresources can also be accessed through REST web services. Consider the following usage notes when working with subresources through REST web services.

You cannot directly delete a subresource. However, it is deleted when the parent record is deleted. Additionally, depending on the record type, you can often set a subrecord's value to null.

You cannot create a subrecord by itself. A subrecord always has to be created in the context of a parent record.

When working with records, you can perform the following operations on subresources:

Get a referenced record that is represented by a select field through the parent record.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/employee/107/subsidiary
```

Get a parent record's subresource: sublist.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder/1/item
```

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder/1/item/1
```

Get a parent record's subresource: subrecord.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder/1/billingaddress
```

Get a parent record's subresource: sublist of a subrecord.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/addressBook/39/addressBookAddress
```

Get a parent record's subresource: sublist of a subrecord of a sublist.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder/6/item/1/inventoryDetail/inventoryAssignment/1
```

Using External IDs

Each record in NetSuite can be uniquely identified by its record type in combination with either an external ID or a system-generated internal ID. For an overview of internal and external IDs, see the help topic [Using Internal IDs, External IDs, and References](#).

You can use an external ID as a key to a record instead of an internal ID. The main use of external IDs is during synchronization with existing data outside of NetSuite.

In REST web services, an external ID starts with the prefix "eid:" in the following format: `eid:external_id`.

An external ID can be any string containing letters, numbers, underscore (_), and hyphen (-).

You can use external IDs anywhere in the URL where an internal ID can be used. You can also use an external ID in the request body with the field name `externalId`.

The following example adds a record. The external ID is used in the request body.

```
1 | POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
2 | BODY {
```

```

3 | "firstName": "John",
4 | "lastName": "Smith",
5 | "isPerson": true,
6 | "externalId": "CID001",
7 | "subsidiary": {
8 |   "id": "1"
9 | }
10 | }

```

The following example retrieves a record. External IDs are used in the request to identify the record.

```
1 | GET http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/eid:CID001/subscriptions/eid:SUID042
```



Tip: If your external ID contains the pipe character (|), this is treated in REST web services as a multi-select delimiter, so the external ID is not parsed correctly. In this case, you can use the following workaround. Use the q= parameter and specify the external ID in the search criteria, surrounded by double quotation marks as in the following example.

```
1 | GET http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/purchaseOrder/?q=externalId IS "RED|P0|1337"
```

Using Datetime Fields

You can work with date and datetime fields using the date and datetime types in NetSuite. See the following sections for information about working with date and datetime types.

When setting a date field using REST web services, you must enter a date or datetime value. You can also specify a time zone in your request. Datetime values are returned in UTC format.

Types of Date-related Fields

The following four types of date-related fields are used in NetSuite.

Type	Example	Notes
Date fields	10/05/2018	Date fields are used, for example, on custom records.
Time fields	10:15 am	Time fields are used, for example, on custom records.
Datetime fields	10/05/2018 10:15 am	Datetime fields are used, for example, on custom records.
Duration fields	25:45 (hours:minutes)	Duration fields are used in time tracking, for example, on time bill records.

Date Field

Date fields use the date element of datetime fields. For example: 2017-07-21. Date field values are not subject to any time zone conversion.

Time Fields

Time fields use the time element of datetime fields, with hours, minutes and second. For example: 17:32:28. Time field values are not subject to any time zone conversion.

Datetime Fields

REST web services use datetime fields in ISO 8601 format and in the UTC time zone. For example: 2017-07-21T17:32:28Z.

Date is the full-date notation as defined by [RFC 3339, section 5.6](#), for example, 2017-07-21.

Date-time is the date-time notation as defined by [RFC 3339, section 5.6](#), for example, 2017-07-21T17:32:28Z.

All datetime values loaded from NetSuite are converted to the UTC time. REST web services accepts datetime values in the same format and in the UTC time zone.

In REST web services, you can also specify the time zone in your request. For example, 2017-07-21T17:32:28+01:00.



Note: During record collection filtering, REST web services uses the underlying N/query SuiteScript module, which uses a different date format. For examples, see [Record Collection Filtering](#).

Duration Fields

Duration fields use the same format as in UI, that is, "hh:mm". For example: "25:42". Duration field values are not subject to any time zone conversion.

Executing Record Actions

REST web services support APIs that provide the programmatic equivalent of clicking a button in the NetSuite user interface. With the record action APIs, you can use REST web services to trigger the same business logic that is triggered by the click of a UI button. Record actions can increase productivity by automating regular tasks that previously had to be done manually in the UI.

By using record actions, you can update the state of a record. Approve and reject are two examples of record actions. When an approve or reject action is executed on a record, the approval status of the record is saved immediately.

REST web services support the same record actions supported by SuiteScript.

For a list of supported actions, see the help topic [Supported Record Actions](#).

Metadata is not provided for record actions.

You can call record actions using the HTTP POST method on the record instance. The action name is prefixed with the string "@".

Action parameters can be passed in the request body in the JSON object.

The following is an example of a record action call with action parameters in the request body.

```
1 POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/vendorPayment/3/@confirm
2 {
3   "confirmationDate": "2019-1-31",
4   "exchangeRate": "4.2",
5   "postingPeriod": 348
6 }
```

You can also execute record actions without providing action parameters, as in the following example.

```
1 | POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/vendorPayment/3/@confirm
```

If an action is performed successfully, the HTTP 200 response is returned.

```
1 | HTTP/1.1 200 OK
2 | Content-Type: application/vnd.oracle.resource+json; type=singular
3 | {
4 |   "links": [
5 |     "self": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/vendorPayment/3/@confirm"
6 |   ],
7 |   "result": true
8 | }
```

When a record action cannot be executed, an HTTP 4xx response is returned.

```
1 | HTTP/1.1 400 Bad Request
2 | Content-Type: application/vnd.oracle.resource; type=error
3 | {
4 |   "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html",
5 |   "title": "Payment already declined",
6 |   "detail": "A payment you are trying to decline is already declined",
7 |   "status": 400,
8 |   "o:errorCode": "USER_ERROR"
9 | }
```

Transforming Records

In REST web services, you can transform a record from one type into another, using data from an existing record. For example, you can create an invoice record from an existing sales order record, using data from the sales order.

All record transformations available in SuiteScript are available in REST web services, too. Transformations are also supported in metadata. For information about the supported transformation types, see the help topic [record.transform\(options\)](#).

Record transformation is generally available for supported records. For a list of fully supported records, you can also see the help topic [REST Web Services Supported Records](#).

In REST web services, you use the POST method to send a record transformation request. The record transformation is executed in a single request. In the request URL, the following details must be specified:

- The record type you that you want to transform.
- The ID of the record that you want to transform.
- The transformation action. The transformation action must be formatted in the following way in the URL: !transform.
- The record type you that you want transform into.

In the following example, a sales order record is transformed into an invoice. If the transformation is performed successfully, an HTTP 204 – No Content response is returned.

```
1 | POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder/3!/transform/invoice
```

In the request body, you can also specify data for additional fields of the record, as shown in the following example.

```
1 | {
```

```

2  "memo": "REST, end date and bills date added",
3  "enddate": "2020-01-14",
4  "billdate": "2020-01-14",
5  "item": {
6    "items": [
7      {
8        "item": { "id": "3456" },
9        "amount": 1
10     }
11   ]
12 }
13 }

```

In the following example, an invoice is transformed into a credit memo. Note that you must specify the doc field, which contains the internal id of the invoice.

```

1  POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/60!transform/creditMemo
2
3  {
4    "toBeEmailed":false,
5    "apply":{
6      "items" :
7      [
8        {
9          "line": 0,
10         "amount":3.01,
11         "apply": true,
12         "applyDate": "2023-09-12",
13         "doc": {"id" : "60"}
14       }
15     ]
16   }
17 }

```

Working with the Pricing Sublist on Item Records

You can create, update, and access pricing sublist (pricing matrix) data through REST web services.

The pricing sublist is available on item records, on the Sales/Pricing subtab.

The pricing sublist is supported if the following features are enabled:

- Multiple Currencies
- Multiple Prices
- Quantity Pricing

For each item, you can set multiple prices based on the following:

- Currency used for item prices. See the help topic [Enabling the Multiple Currencies Feature](#).
- Price levels available for the item, as defined in the price level user-defined list. See the help topic [Using Multiple Pricing](#).
- The quantity of items being sold, when the Quantity Pricing feature is enabled. See the help topic [Using Quantity Pricing](#).

For general information about item pricing, see the following help topics:

- [Item Pricing](#)
- [Setting Up Item Pricing](#)
- [Using Item Records](#)

In REST web services, you can create or update price values on a per-row, per-column basis by using the POST operation. You can access the price values on a per-row, per-column basis by using the GET operation.

To create or update a pricing sublist, you send a POST request similar to the following and specify the pricing sublist data in the request body in an array of items:

```
1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryitem
2 "price": {
3   "items": [
4     {
5       "currencyPage" : { "id" : "1" },
6       "priceLevel" : { "id" : "1" },
7       "quantity" : { "value" : 5 },
8       "price": 25.5
9     },
10    ...
11  ]
12 }
```

To access a pricing sublist, you send a GET request similar to the following:

```
1 GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryitem/482/price?expandSubResources=true
```

Using the REST Web Services SuiteScript Execution Context

Execution contexts provide information about how or when a script or workflow is triggered to execute. For example, a script can be triggered in response to an action in the NetSuite application or an action occurring in another context, such as a web services integration. You can use execution context filtering to ensure that your scripts or workflows are triggered only when necessary. This filtering can improve performance in contexts where the scripts or workflows are not required.

For NetSuite user event and client scripts, you can set up execution context filtering to control the contexts in which these scripts can run. You can set up execution context filtering in the Execution Context field on the Context Filtering subtab of the Script Deployment record. For information about script deployment, see the help topic [Methods of Deploying a Script](#). For more information about execution contexts in general, see the help topics [Execution Contexts](#) and [runtime.executionContext](#).

If a script has its execution context set to REST web services, the script only executes if it is triggered through a REST web services request.

For example, you can deploy a script similar to the following on customer records. In this example, the `beforeSubmit` script is only executed on the record if the request is triggered through REST web services.

```
1 define(['N/record', 'N/runtime'],
2   function(record, runtime) {
3     function beforeSubmit(context) {
4       log.debug("Before submit of " + context.newRecord.type);
5       var executionContext = runtime.executionContext;
6       var isRestWs = (runtime.ContextType.REST_WEBSERVICES === executionContext);
7       log.debug("Is originating from REST WS: " + isRestWs);
8     }
9   })
```

After running the script, you can see its execution context in the script execution log. See the help topic [Using the Script Execution Log Subtab](#). You can also find information about the execution context in the system notes of a record. See the help topic [Viewing System Notes on a Record](#).

Working with Sublists

Sublists are an important part of several NetSuite record types. In REST web services, you can use the POST and PATCH operations to access a sublist on a record instance. However, because sublists in NetSuite are of multiple types, for instance, editable sublist, applicable sublist, or search sublist, each sublist can have slightly different behavior. Some sublists on particular records can even contain pre-generated lines. Moreover, if a line is added based on user input, it can contain default or computed values. Some sublists include one or more key fields that can uniquely identify each sublist record. These are keyed sublists. The behavior of keyed sublists is different from the behavior of non-keyed sublists.

Note: For information about the format of sublists in REST web services, see [Format of Sublists and Subrecords](#).

The following list includes all operations that can be performed with a sublist:

1. Add new
 - Single line
 - Multiple lines at once
2. Update existing
 - Single line
 - Unset particular field
 - Multiple lines at once
3. Remove existing
 - Single line – The delete operation is not available for removing single lines. You can delete a single line by using the patch operation, as in the following example:

```

1 | PATCH /record/v1/purchaseOrder/:id?replace=item
2 | {
3 |   „item“: {
4 |     „items“: [
5 |       { . all lines except those you want to delete . }
6 |     ]
7 |   }
8 | }
```

- Multiple lines at once
 - All
4. Mix (any combination of the above at once)
 5. Replace all

Operations 2.a, 2.b, 2.c, 3.a, 3.b, and 4 are not possible on non-keyed sublists. You can achieve the equivalent of these operations by replacing all sublist lines.

SOAP web services offer a special **replaceAll** attribute to enable some of these operations. For more information about this topic, see the help topic [Updating Sublists in SOAP Web Services](#).

Similar behavior is also reflected in REST web services. This section provides details about each sublist operation mode.

In all the examples in this section, assume the following initial state of a record instance (unless otherwise specified).

1 | GET .../myrecord/100

```

2  {
3
4      "body1": "previous body text 1",
5      "body2": "previous body text 2",
6      "sublist" : {
7          "items" : [
8              { "key1": "a", "key2": "1", "col": "previously present line 1" },
9              { "key1": "b", "key2": "2", "col": "previously present line 2" },
10             { "key1": "X", "key2": "0", "col": "previously present line 0" },
11         ]
12     },
13     "unkeyedsublist" : {
14         "items" : [
15             { "col": "previously present line 1" },
16             { "col": "previously present line 2" },
17             { "col": "previously present line 0" },
18         ]
19     }
20 }

```

See the following topics for more details:

- [Creating a Sublist](#)
- [Updating a Sublist](#)
- [Replacing a Sublist](#)
- [Deleting a Sublist](#)

Creating a Sublist

During create operations, lines of a keyed sublist are updated if the line contains matching keys, and the remaining lines from the request are added. For non-keyed sublists, the lines are always appended to the ones that are already present on the sublist by default. The following is an example of a POST operation upon a sublist.

Request Body

POST .../myrecord

```

1  {
2      "body1": "inserted body text 1",
3      "sublist" : {
4          "items" : [
5              { "key1": "a", "key2": "1", "col": "inserted line 1" },
6              { "key1": "b", "key2": "2", "col": "inserted line 2" }
7          ]
8      },
9      "unkeyedsublist" : {
10         "items" : [
11             { "col": "inserted line 1" },
12             { "col": "inserted line 2" }
13         ]
14     }
15 }

```

Response – Post State

Response:

201 Created

Location: .../myrecord/101

```

1  {
2    "body1": "inserted body text 1",
3    "body2": "default body text 2",
4    "sublist" : {
5      "items" : [
6        { "key1": "a", "key2": "1", "col": "inserted line 1" },
7        { "key1": "b", "key2": "2", "col": "inserted line 2" },
8        { "key1": "X", "key2": "0", "col": "default line 0" },
9      ]
10   },
11   "unkeyedsublist" : {
12     "items" : [
13       { "col": "default line 1" },
14       { "col": "default line 2" },
15       { "col": "default line 0" },
16       { "col": "inserted line 1" },
17       { "col": "inserted line 2" },
18     ]
19   }
20 }

```



Note: Keyed sublist lines are updated if the key matches, otherwise, a new line is created (in other words, sublist lines are merged). Lines for non-keyed sublists are added.

Updating a Sublist

During update operations, lines of a keyed sublist are updated if the line contains matching keys, and the remaining lines from the request are added. For non-keyed sublists, the lines are always appended to the ones that are already present on the sublist. The following examples show PATCH operations upon a sublist. Key 1 and Key 2 indicate the field ID and value, respectively.

Request Body – Patch Operation

PATCH .../myrecord/100

```

1  {
2    "body1": "replaced body text 1",
3    "sublist" : {
4      "items" : [
5        { "key1": "a", "key2": "1", "col": "replaced line 1" },
6        { "key1": "b", "key2": "2", "col": "replaced line 2" }
7      ]
8    },
9    "unkeyedsublist" : {
10     "items" : [
11       { "col": "inserted line 1" },
12       { "col": "inserted line 2" }
13     ]
14   }
15 }

```

Response – Post State

Response:

204 No Content

Post State:

```

1 {
2   "body1": "replaced body text 1",
3   "body2": "previous body text 2",
4   "sublist" : {
5     "items" : [
6       { "key1": "a", "key2": "1", "col": "replaced line 1" },
7       { "key1": "b", "key2": "2", "col": "replaced line 2" },
8       { "key1": "X", "key2": "0", "col": "previously present line 0" },
9     ]
10  },
11  "unkeyedsublist" : {
12    "items" : [
13      { "col": "previously present line 1" },
14      { "col": "previously present line 2" },
15      { "col": "previously present line 0" },
16      { "col": "inserted line 1" },
17      { "col": "inserted line 2" },
18    ]
19  }
20 }

```



Note: Keyed sublist lines are updated if the key matches, otherwise, a new line is created (in other words, sublist lines are merged). All lines of non-keyed sublists are added.

Request Body – Nullifying a Sublist

PATCH ../myrecord/100

```

1 {
2   "body1": "replaced body text 1",
3   "sublist" : {
4     "items" : null
5   }
6 }

```

```

1 {
2   "body1": "replaced body text 1",
3   "sublist" : null
4 }

```

Response – Post State

Response:

204 No Content

Post State:

```

1 {
2   "body1": "replaced body text 1",
3   "body2": "previous body text 2",
4   "sublist" : {
5     "items" : []
6   },
7   "unkeyedsublist" : {
8     "items" : [
9       { "col": "previously present line 1" },
10      { "col": "previously present line 2" },
11      { "col": "previously present line 0" },
12    ]
13  }

```

14 | }



Note: Keyed as well as non-keyed sublist lines are replaced. If the sublist is mandatory, it is expected that the operation fails.

Replacing a Sublist

During sublist replacements, lines of a keyed sublist are updated if the line contains matching keys, other lines are removed, and the remaining unmatched lines from the request are added. For non-keyed sublists, all lines are always removed and replaced by lines that are in the incoming request. The following examples show POST and PATCH operations with a replace query parameter.

Request Body – Replacing a Default Keyed Sublist

POST .../myrecord?replace=sublist

```

1  {
2    "body1": "inserted body text 1",
3    "sublist" : {
4      "items" : [
5        { "key1": "a", "key2": "1", "col": "inserted line 1" },
6        { "key1": "b", "key2": "2", "col": "inserted line 2" }
7      ]
8    },
9    "unkeyedsublist" : {
10     "items" : [
11       { "col": "inserted line 1" },
12       { "col": "inserted line 2" }
13     ]
14   }
15 }
```

Response – Post State

Response:

201 Created

Location: .../myrecord/101

```

1  {
2    "body1": "inserted body text 1",
3    "body2": "default body text 2",
4    "sublist" : {
5      "items" : [
6        { "key1": "a", "key2": "1", "col": "inserted line 1" },
7        { "key1": "b", "key2": "2", "col": "inserted line 2" },
8      ]
9    },
10   "unkeyedsublist" : {
11     "items" : [
12       { "col": "default line 1" },
13       { "col": "default line 2" },
14       { "col": "default line 0" },
15       { "col": "inserted line 1" },
16       { "col": "inserted line 2" },
17     ]
18   }
19 }
```

Note: Keyed sublist lines are updated if the key matches, other lines are removed, and unmatched lines from the request are created as new. All lines of non-keyed sublists are added. The operation fails if any replaced default sublist line is read-only.

Request Body – Replacing a Default Non-Keyed Sublist

POST .../myrecord?replace=unkeyedsublist

```

1 {
2   "body1": "inserted body text 1",
3   "sublist" : {
4     "items" : [
5       { "key1": "a", "key2": "1", "col": "inserted line 1" },
6       { "key1": "b", "key2": "2", "col": "inserted line 2" }
7     ]
8   },
9   "unkeyedsublist" : {
10    "items" : [
11      { "col": "inserted line 1" },
12      { "col": "inserted line 2" }
13    ]
14  }
15 }
```

Response – Post State

Response:

201 Created

Location: .../myrecord/101

```

1 {
2   "body1": "inserted body text 1",
3   "body2": "default body text 2",
4   "sublist" : {
5     "items" : [
6       { "key1": "a", "key2": "1", "col": "inserted line 1" },
7       { "key1": "b", "key2": "2", "col": "inserted line 2" },
8       { "key1": "X", "key2": "0", "col": "default line 0" },
9     ]
10  },
11  "unkeyedsublist" : {
12    "items" : [
13      { "col": "inserted line 1" },
14      { "col": "inserted line 2" },
15    ]
16  }
17 }
```

Note: Keyed sublist lines are updated if the key matches, otherwise, a new line is created. All lines of non-keyed sublists are removed and then created as new. The operation fails if any replaced default sublist line is read-only.

Request Body – Replacing a Keyed Sublist

PATCH .../myrecord/100?replace=sublist

```

1 {
2   "body1": "replaced body text 1",
3   "sublist" : {
4     "items" : [
5       { "key1": "a", "key2": "1", "col": "replaced line 1" },
6       { "key1": "b", "key2": "2", "col": "replaced line 2" }
7     ]
8   },
9   "unkeyedsublist" : {
10    "items" : [
11      { "col": "inserted line 1" },
12      { "col": "inserted line 2" }
13    ]
14  }
15 }

```

Response – Post State

Response:

204 No Content

Post State:

```

1 {
2   "body1": "replaced body text 1",
3   "body2": "previous body text 2",
4   "sublist" : {
5     "items" : [
6       { "key1": "a", "key2": "1", "col": "replaced line 1" },
7       { "key1": "b", "key2": "2", "col": "replaced line 2" },
8     ]
9   },
10  "unkeyedsublist" : {
11    "items" : [
12      { "col": "previously present line 1" },
13      { "col": "previously present line 2" },
14      { "col": "previously present line 0" },
15      { "col": "inserted line 1" },
16      { "col": "inserted line 2" },
17    ]
18  }
19 }

```



Note: Keyed sublist lines are updated if the key matches, other lines are removed, and unmatched lines from the request are created as new. All lines of non-keyed sublists are added.

Request Body – Replacing a Non-Keyed Sublist

PATCH ../myrecord/100?replace=unkeyedsublist

```

1 {
2   "body1": "replaced body text 1",
3   "sublist" : {
4     "items" : [
5       { "key1": "a", "key2": "1", "col": "replaced line 1" },
6       { "key1": "b", "key2": "2", "col": "replaced line 2" }
7     ]
8   },
9   "unkeyedsublist" : {
10    "items" : [
11      { "col": "inserted line 1" },
12      { "col": "inserted line 2" }
13    ]
14  }
15 }

```

```

13 |     ]
14 |   }
15 | }

```

Response – Post State

Response:

204 No Content

Post State:

```

1 | {
2 |   "body1": "replaced body text 1",
3 |   "body2": "previous body text 2",
4 |   "sublist" : {
5 |     "items" : [
6 |       { "key1": "a", "key2": "1", "col": "replaced line 1" },
7 |       { "key1": "b", "key2": "2", "col": "replaced line 2" },
8 |       { "key1": "X", "key2": "0", "col": "previously present line 0" },
9 |     ]
10 |   },
11 |   "unkeyedsublist" : {
12 |     "items" : [
13 |       { "col": "inserted line 1" },
14 |       { "col": "inserted line 2" },
15 |     ]
16 |   }
17 | }

```



Note: Keyed sublist lines are updated if the key matches, otherwise, a new line is created (in other words, sublist lines are merged). All lines of non-keyed sublists are removed and then created as new.

Request Body – Replacing Multiple Sublists

The following operation is a combination of the previous operations.

PATCH ../myrecord/100?replace=sublist,unkeyedsublist

```

1 | {
2 |   "body1": "replaced body text 1",
3 |   "sublist" : {
4 |     "items" : [
5 |       { "key1": "a", "key2": "1", "col": "replaced line 1" },
6 |       { "key1": "b", "key2": "2", "col": "replaced line 2" }
7 |     ]
8 |   },
9 |   "unkeyedsublist" : {
10 |    "items" : [
11 |      { "col": "inserted line 1" },
12 |      { "col": "inserted line 2" }
13 |    ]
14 |   }
15 | }

```

Response – Post State

Response:

204 No Content

Post State:

```

1 | {
2 |   "body1": "replaced body text 1",
3 |   "body2": "previous body text 2",
4 |   "sublist" : {
5 |     "items" : [
6 |       { "key1": "a", "key2": "1", "col": "replaced line 1" },
7 |       { "key1": "b", "key2": "2", "col": "replaced line 2" },
8 |     ]
9 |   },
10 |   "unkeyedsublist" : {
11 |     "items" : [
12 |       { "col": "inserted line 1" },
13 |       { "col": "inserted line 2" },
14 |     ]
15 |   }
16 | }

```

Deleting a Sublist

You can delete a sublist, that is, remove all the lines of a sublist, by replacing it with an empty sublist. To delete a sublist, you send a PATCH request with the `replace` query parameter. The following example shows how to delete a sublist.

The operation works both for keyed and non-keyed sublists. The sublist must be specified both in the request body and the query parameter.

If the sublist is mandatory, the operation fails.

Request Body

PATCH .../myrecord?replace=sublist

```

1 | {
2 |   "sublist" : {
3 |     "items" : []
4 |   }
5 | }

```

Alternatively, you can delete a sublist by setting the value of the sublist to `null`.

PATCH .../myrecord?replace=sublist

```

1 | {
2 |   "sublist" : null
3 | }

```

Response

Response: 204 No Content

Post State:

```

1 | {

```

```
2  "body1": "replaced body text 1",
3  "body2": "previous body text 2",
4  "sublist" : {
5      "items" : []
6  },
7  "unkeyedsublist" : {
8      "items" : [
9          { "col": "previously present line 1" },
10         { "col": "previously present line 2" },
11         { "col": "previously present line 0" },
12     ]
13 }
14 }
```

Working with Subrecords

Subrecords represent a way of storing data in NetSuite. Like records, subrecords are classified by type. Some common types of subrecord include address, inventory detail, and order schedule.

A subrecord includes many of the same elements of a standard NetSuite record (body fields, sublists and sublist fields, and so on). However, subrecords must be created, edited, removed, or viewed from within the context of a standard (parent) record.

The purpose of a subrecord is to hold key related data about the parent record. For example, a parent record would be a Serialized Inventory Item record. This record defines a type of item. A subrecord would be an Inventory Detail subrecord. This is a subrecord that contains all data related to where the item might be stored in a warehouse. In this way, the subrecord contains data related to the item, but not data that directly defines the item. Without the parent record, the subrecord would serve no purpose. For more information about subrecords in general, see the help topic [About Subrecords](#).

In REST web services, you can use the POST and PATCH operations to access a subrecord on the record instance. Every operation on a subrecord must be done in the context of the parent record. For example, you cannot create a subrecord without creating a parent record.

Note: For information about the format of subrecords in REST web services, see [Format of Sublists and Subrecords](#).

Updating a Subrecord

Subrecords are modelled as inner JSON properties in the request and response body. In the following example, a subrecord, addressbookaddress is set on the sublist line of the addressbook sublist.

```
1 POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
2 {
3   "entityid": "My Customer",
4   "companyname": "My Company",
5   "email": "another.customer@example.com",
6   "addressbook": {
7     "items": [{
8       "label": "New York HQ",
9       "addressbookaddress": {
10        "country": "US",
11        "state": "NY",
12        "zip": "10001",
13        "addressee": "Dwight Schrute"
14      }
15    }]
16  }
17 }
```

Getting a Subrecord

The following example shows a GET operation, which is used to access a subrecord.

```
1 GET http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/42/addressbook/24/addressbookaddress
2 {
3   "addressee": "Dwight Schrute",
4   "city": "New York",
5   "country": "US",
6   "dropdownstate": {
```

```
7      "id": "NY",  
8      "refName": "New York",  
9      ...  
10     },  
11     "state": "NY",  
12     "zip": "10001",  
13     ...  
14 }
```

The structure of the subrecord is also returned on the parent record, on the sublist line, if you set the `expandSubResources` query parameter to `true`.

Record Filtering and Query

The query operation is used to execute a query on a specific record type based on a set of criteria.

Record query only returns record IDs and HATEOAS links. That is, query results have a form of non-expanded references. Additionally, you can only use body fields in query conditions. Saved queries, multilevel joins, and sublist and subrecord queries are not supported.

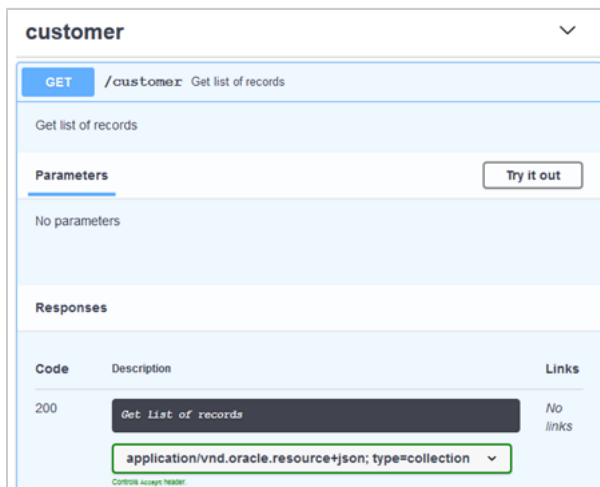
REST web services only supports limited record query. Joins are not supported.

For more information, see the following topics:

- [Listing All Record Instances](#)
- [Record Collection Filtering](#)
- [Executing SuiteQL Queries Through REST Web Services](#)
- [Working with SuiteAnalytics Datasets in REST Web Services](#)
- [Collection Paging](#)

Listing All Record Instances

You can obtain the list of all records of a record type by sending an HTTP GET request to `.../services/rest/record/v1/record_type`, as shown in the following image.



Listing all instances of a given record type can also be understood as a query over the given record type without any criteria. The following is an example of such a request.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
```

The following is a shortened example of a response.

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=1000&offset=0"
6 |     }
7 |   ],
8 |   "items": [
```

```

9      {
10         "links": [
11             {
12                 "rel": "self",
13                 "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
14             }
15         ],
16         "id": "107"
17     },
18     {
19         "links": [
20             {
21                 "rel": "self",
22                 "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/41"
23             }
24         ],
25         "id": "41"
26     },
27     {
28         "links": [
29             {
30                 "rel": "self",
31                 "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/90"
32             }
33         ],
34         "id": "90"
35     }
36 ],
37 "totalResults": 3
38 }

```

Record Collection Filtering

You can filter the collection of all record instances by using the `q` query parameter to specify filter conditions. Each condition consists of a field name, an operator, and a value. You can join several conditions using the AND / OR logical operators, and you can use `()` to mark precedence.

REST web services uses the `N/query` module to perform record collection filtering. Before filtering collections in REST web services, see the help topic [N/query Module](#) for information about how this module works.

The following table contains the list of available query operators with their associated field types.

Field Type	Allowed Filters
None	EMPTY, EMPTY_NOT
Boolean	IS, IS_NOT
Double, Integer, Float, Number, Duration	ANY_OF, ANY_OF_NOT, BETWEEN, BETWEEN_NOT, EQUAL, EQUAL_NOT, GREATER, GREATER_NOT, GREATER_OR_EQUAL, GREATER_OR_EQUAL_NOT, LESS, LESS_NOT, LESS_OR_EQUAL, LESS_OR_EQUAL_NOT, WITHIN, WITHIN_NOT
String	CONTAIN, CONTAIN_NOT, IS, IS_NOT, START_WITH, START_WITH_NOT, ENDWITH, END_WITH_NOT
Date / Time	AFTER, AFTER_NOT, BEFORE, BEFORE_NOT, ON, ON_NOT, ON_OR_AFTER, ON_OR_AFTER_NOT, ON_OR_BEFORE, ON_OR_BEFORE_NOT

Not all operators accept one value. Some operators do not require any value, some operators require two values, and some operators accept any number of values. Consider the following examples:

- **Unary operators:** The `EMPTY` and `EMPTY_NOT` operators do not accept any values. For example: `?q=companyName EMPTY`

- Ternary operators: The BETWEEN, BETWEEN_NOT, WITHIN, and WITHIN_NOT operators accept two values. For example: ?q=id BETWEEN_NOT [1, 42]
- N-ary operators: The ANY_OF and ANY_OF_NOT operators do accept one or any higher number of values. For example: ?q=id ANY_OF [1, 2, 3, 4, 5]

You can find the fields available for filtering in the metadata. Fields you can filter by have the x-ns-filterable parameter. For information about metadata, see [Working with Resource Metadata](#).

For information about using datetime fields in queries, see [Using Datetime Fields](#).

Note: During record collection filtering, REST web services uses the underlying N/query SuiteScript module, which uses a different date format than the formats outlined in [Using Datetime Fields](#).

The following is an example of a simple query.

Note: The spaces in URLs are encoded. The following examples are presented without encoding for clarity.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?q=email START_WITH barbara
```

The response is a collection of customer record instances where the value in the email field starts with the value of barbara. The result is a collection resource containing links to resources that match query criteria. The response could be similar to the following:

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=1000&offset=0"
6 |     }
7 |   ],
8 |   "items": [
9 |     {
10 |      "links": [
11 |        {
12 |          "rel": "self",
13 |          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
14 |        }
15 |      ],
16 |      "id": "107"
17 |    },
18 |   ],
19 |   "totalResults": 1
20 | }
```

When your condition value contains spaces, you should use quotation marks around the constraint, for instance, `firstname IS "Barbara Allen"`. See the following additional query examples:

- Find customer by company name (string value):
GET `https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?q=companyname START_WITH "Another Company"`
- Find inactive customers (boolean value):
GET `https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?q=isinactive IS true`
- Find customers created in 2019 (date value, AND operator):
GET `https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?q=dateCreated ON_OR_AFTER "1/1/2019" AND dateCreated BEFORE "1/1/2020"`

- Find customers with high or low credit limit (number constraint, OR operator):

GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?q=creditlimit GREATER_OR_EQUAL 1000 OR creditlimit LESS_OR_EQUAL 10



Note: When you join more than one conditions, the AND logical operator has priority over the OR logical operator. To change this behavior, use parentheses as follows: "(" and ")".

Executing SuiteQL Queries Through REST Web Services

SuiteQL is a query language based on the SQL database query language. SuiteQL provides advanced dynamic query capabilities that can be used to access NetSuite records.

For more information about SuiteQL in general, see the help topic [SuiteQL](#).

You can execute SuiteQL queries through REST web services by sending a POST request to the `suiteql` resource, and specifying the query in the request body after the body parameter `q`.

In the request URL you can also specify the number of results you want to return in a single page and the page offset. For information about paging and offset values, see [Collection Paging](#).

You can use the Records Catalog to get information about all record types and fields that are available through the analytics data source. The browser includes a page for each record type with all available fields and joined record types, if applicable. For each field, you can also find whether that field is available for use in SuiteAnalytics Workbook, SuiteScript, SuiteTalk REST web services, and SuiteAnalytics Connect. For more information, see the help topic [Records Catalog Overview](#).

Using SuiteQL queries, you can return a maximum of 100,000 results. If you expect your queries to return more than 100,000 results, you can use SuiteAnalytics Connect with the NetSuite2.com data source. For more information about how to use SuiteAnalytics Connect, see the help topic [Getting Started with SuiteAnalytics Connect](#).

For more information, see the help topic [query.runSuiteQLPaged\(options\)](#).

The following example shows a SuiteQL query executed through REST web services. Note that `Prefer: transient` is a required header parameter.

```
1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=10&offset=10
2 Prefer: transient
3 {
4   "q": "SELECT email, COUNT(*) as count FROM transaction GROUP BY email"
5 }
```

The following is a shortened response.

```
1 {
2   "links": ...,
3   "count": 3,
4   "offset": 10,
5   "totalResults": 53,
6   "items": [
7     {
8       "links": [],
9       "email": "test@netsuite.com",
10      "count": "143"
11    },
12    {
13      "links": [],
```

```

14 |     "email": "test2@netsuite.com",
15 |     "count": "144"
16 |   },
17 |   {
18 |     "links": [],
19 |     "email": "test3@netsuite.com",
20 |     "count": "145"
21 |   }
22 | ],
23 | ...
24 | }

```

In SuiteQL queries, you can also use advanced SQL functions, for example joins and concatenations. The following example shows a request where the first and last names from employee records are concatenated as full name.

```

1 | {
2 |   "q": "SELECT CONCAT(firstname,lastname) as fullname FROM employee"
3 | }

```

When working with SuiteQL, consider the limitation listed at [SuiteQL Limitations and Exceptions](#).

Working with SuiteAnalytics Datasets in REST Web Services

In SuiteAnalytics Workbook, you analyze your company data using two distinct objects: a dataset and a workbook.

Datasets are the basis for all workbooks and workbook components in your account. In a dataset, you combine record type fields and criteria filters to create a query. The results of this query act as the source data for your workbooks.

Prior to 2020.1, you defined, visualized, and analyzed SuiteAnalytics Workbook source data in a single workbook object.

As of 2020.1, workbooks and datasets exist as separate objects, each with their own respective owners and accessibility rights. You define a dataset by selecting record types, fields, and criteria.

REST web services support the execution of datasets. Dataset execution replaces the previously available workbook execution function. You can execute both standard and custom datasets by using the dataset ID in your GET request.

When you work with datasets in REST web services, consider the following:

- You must have the SuiteAnalytics Workbook feature enabled. See [Prerequisites and Setup for REST Web Services](#).
- The SuiteAnalytics Workbook permission must be assigned to the user's role. See [Prerequisites and Setup for REST Web Services](#).
- Datasets must be created in the SuiteAnalytics Workbook UI and cannot be created through REST web services. For information about creating datasets, see the help topic [Defining a Dataset](#).
- Metadata is not provided for datasets.

Executing Datasets Through REST Web Services

To execute a dataset, you send a GET request similar to the following:

1 | GET <https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/dataset/ExampleDataset1/result>

The following is the response. The response is in a key-value structure or table.

Note: REST web services do not return links to transaction records. For other record types, you must set the internal ID column for the dataset in the UI. If the internal ID is set, REST web services returns a link to non-transaction records.

```

1  {
2    "links": [
3      {
4        "rel": "self",
5        "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/dataset/ExampleDataset1/result?
limit=1000&offset=0"
6      }
7    ],
8    "items": [
9      {
10       "links": [],
11       "amount": "5.33333334",
12       "amount2": "2",
13       "entity": "Customer AU",
14       "item": "Item Tax 2 Lbs(AU)",
15       "itemcount": "1",
16       "postingperiod": "May 2010",
17       "trandate": "5/28/2010"
18     },
19   ],
20   "count": 1,
21   "hasMore": false,
22   "offset": 0,
23   "totalResults": 1
24 }

```

If you use formulas in datasets, when you execute a dataset in REST, the user-defined name of the formula is returned as the field ID for the formula field. You can define the formula field names when you define a dataset in the SuiteAnalytics Workbook UI. For information about defining datasets, see the help topic [Defining a Dataset](#). For information about creating formula fields, see the help topic [Creating Formula Fields in SuiteAnalytics Workbook](#).

The following example shows that the year_created formula name is returned as the ID for the formula field.

```

1  {
2    ...
3    "items": [
4      {
5        "companyname": "Glenrock General Hospital",
6        "email": "alan.rath@ns.com",
7        "year_created": "2012"
8      },
9      {
10       "companyname": "Customer CyberSource",
11       ...
12     }
13   ],
14   "totalResults": 2
15 }

```

When executing datasets through REST web services, consider the following:

- The response from a dataset execution returned through REST web services may have different format and values than responses for record instance requests.

- A REST web services request executes a single dataset. You cannot execute multiple datasets through REST web services at the same time.

Getting a List of Datasets Through REST Web Services

You can list all existing SuiteAnalytics Workbook datasets through REST web services. Datasets must be created in the SuiteAnalytics Workbook UI and cannot be created through REST web services. For information about creating datasets, see the help topic [Defining a Dataset](#).

To list all existing datasets, you send a GET request similar to the following:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/dataset/
```

The result contains the name, ID, and description of the datasets, and it provides information about the record type the dataset works on.

You can also use collection paging when you list all datasets. To list all existing datasets and page through the results, you send a GET request similar to the following:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/dataset/?limit=10&offset=10
```

The request returns 10 items on the list, starting by the 11th item. For information about paging, see [Collection Paging](#).

Collection Paging

Lists of record instances and the results of saved analytics workbook searches are returned in one or more pages. The results are displayed on multiple pages, with the default setting of 1000 results per page on a maximum of 1000 pages. This way you can return maximum one million results if you use SuiteAnalytics Connect. Without SuiteAnalytics Connect, you can display maximum 100 000 results.

You can also specify your own paging by adding the limit criteria to your request.

Note: You can specify your criteria to return fewer than 1000 results per page. In this case, the overall number of returned results is lower, too, because a maximum of 1000 pages of results can be returned.

You can achieve more results by using the ODBC connector. For information, see the help topic [Accessing the Connect Service Using an ODBC Driver](#).

The following is an example of a request for all instances of customer records, displaying two results per page:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2
```

The following is an example of a response:

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "next",
5 |       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account?limit=2&offset=2"
6 |     },
7 |   ]
8 | }
```

```

8      "rel": "last",
9      "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account?limit=22&offset=2"
10    },
11    {
12      "rel": "self",
13      "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=0"
14    }
15  ],
16  "items": [
17    {
18      "links": [
19        {
20          "rel": "self",
21          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
22        }
23      ],
24      "id": "107"
25    },
26    {
27      "links": [
28        {
29          "rel": "self",
30          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/41"
31        }
32      ],
33      "id": "41"
34    }
35  ],
36  "count": 2,
37  "offset": 0,
38  "hasMore": true,
39  "totalResults": 3
40 }

```

The values of the count, hasMore, and totalResults fields provide information about the results that can be retrieved in subsequent requests for other pages. Links to the last and next pages provide direct links that you can use to retrieve these pages.

You can retrieve the next page by sending a request with the offset value specified. The offset provides the index of the result from which you should start the requested page.

Note: You can only set offset and limit values in a way that the offset is divisible by the limit. For example, Offset=20, Limit=10 or Offset=0, Limit=5

The following is an example of a request for the second (and last) page of results with all customers:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account?limit=2&offset=2
```

The following is an example of a response:

```

1  {
2    "links": [
3      {
4        "rel": "prev",
5        "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=0"
6      },
7      {
8        "rel": "first",
9        "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=22&offset=0"
10     },
11     {
12       "rel": "self",
13       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=2"
14     }
15   ],
16   "items": [
17     {

```

```
18     "links": [  
19         {  
20             "rel": "self",  
21             "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/90"  
22         }  
23     ],  
24     "id": "90"  
25 }  
26 ],  
27 "count": 1,  
28 "offset": 2,  
29 "hasMore": false,  
30 "totalResults": 3  
31 }
```

Error Handling and Logging in REST Web Services

For information about error handling and logging, see the following topics:

- [Error Handling in REST Web Services](#)
- [Using the REST Web Services Execution Log](#)

Error Handling in REST Web Services

In REST web services, HTTP status codes are used to inform you about the success or failure of a request. The following status codes are used:

- 2xx status codes are used for successful requests.
- 4xx status codes are used for failures due to user error.
- 5xx status codes are used for failures due to system error.

The error messages clarify whether an error occurred in the request URL, the request headers, the query parameters, or the request body. Error messages also provide more information about the cause of errors and provide possible solutions. Error responses contain the following fields.

- status – the HTTP status code.
- o:errorDetails – an array with error details. One error response can contain multiple error details.
 - detail – human-readable error description.
 - o:errorCode – the error category in a machine-friendly format.
 - o:errorPath (optional) – a machine-readable JSON path that points to the request body where the error occurred.
 - o:urlPath (optional) – a machine-readable URI path that points to the part of the URL where the error occurred.
 - o:errorHeader (optional) – the machine-friendly name of the request header where the error occurred.
 - o:errorQueryParam (optional) – the machine-friendly name of the request query parameter where the error occurred.

The following sections show some common errors with their descriptions.

Invalid Login Information

The following error is returned if the request contains an invalid login token.

```

1 {
2   "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html",
3   "title": "Unauthorized",
4   "status": 401,
5   "o:errorDetails": [
6     {
7       "detail": "Invalid login attempt. For more details, see the Login Audit Trail in the NetSuite UI at Setup > Users/Roles > User Management > View Login Audit Trail.",
8       "o:errorCode": "INVALID_LOGIN"
9     }
10  ]
11 }
```

```

9      }
10     ]
11  }

```

Request with an Invalid Body

The following is a POST request <https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder> with an invalid reference to a sublist item in the request body.

```

1  {
2    "entity": { "id": 107 },
3    "location": { "id": 1 },
4    "item": {
5      "items": [
6        {
7          "item": { "id": 9999 },
8          "amount": 1
9        }
10     ]
11  }
12 }

```

The request returns the following error with the JSON path `o:errorPath` pointing to the invalid value.

```

1  {
2    "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html#sec10.4.1",
3    "title": "Bad Request",
4    "status": 400,
5    "o:errorDetails": [
6      {
7        "detail": "Error while accessing resource: You have entered an Invalid Field Value 9999 for the following field: item",
8        "o:errorCode": "INVALID_CONTENT",
9        "o:errorPath": "item.items[0].item"
10     }
11  ]
12 }

```

Request for a Non-existent Record

The request GET <https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/999> returns the following error if the request is for a non-existent customer.

```

1  {
2    "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html",
3    "title": "Not Found",
4    "status": 404,
5    "o:errorDetails": [
6      {
7        "detail": "Record does not exist.",
8        "o:errorCode": "NONEXISTENT_ID"
9      }
10  ]
11 }

```

Invalid Request

The request GET <https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/customer, salesOrder> returns the following error because there is an extra space in the query

parameters. The correct request is `https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog?select=customer,salesOrder`.

```

1 {
2   "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html",
3   "title": "Bad Request",
4   "status": 400,
5   "o:errorDetails": [
6     {
7       "detail": "The request could not be understood by the server due to malformed syntax.",
8       "o:errorCode": "INVALID_REQUEST"
9     }
10  ]
11 }

```

Exceeded Concurrency Governance Limit

The following error is returned if the request is rejected due to exceeding the limit allowed by concurrency governance. For information about request limits, see [Concurrency Governance and Session Management](#).

If this error occurs, retry sending the request. For information about implementing a retry logic in your code, see the help topic [Retrying Failed Web Services Requests](#).

```

1 {
2   "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html",
3   "title": "Bad Request",
4   "status": 429,
5   "o:errorCode": "USER_ERROR"
6   "o:errorDetails": [
7     {
8       "detail": "Concurrent request limit exceeded. Request blocked.",
9       "o:errorCode": "CONCURRENCY_LIMIT_EXCEEDED"
10    }
11  ]
12 }

```

System Error

The following error is returned if a system error occurs while the request is being processed. If a system error occurs, contact NetSuite Customer Support and refer to the error ID.

```

1 {
2   "type": "https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html",
3   "title": "Internal Server Error",
4   "status": 500,
5   "o:errorDetails": [
6     {
7       "detail": "An unexpected error occurred. Error ID: jrgbpyylphhishmmlxyt",
8       "o:errorCode": "UNEXPECTED_ERROR"
9     }
10  ]
11 }

```

Request Headers that Modify the Error Behavior

You can use two different request headers to modify the way errors are handled. You must send the headers in the following format:

```

1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
2 X-NetSuite-PropertyNameValidation: error
3 {
4   ...
5 }

```

PropertyNameValidation

The possible values of this request header are ignore, warning, and error. The default value is warning.

This is a behavior for invalid field names in the request body, such as field names with incorrect spelling, or field names that do not exist on the record. Using this header returns an error if the field in the request is invalid, for example, if it does not exist.

Warnings are returned in the response headers.

PropertyValueValidation

The possible values of this request header are: ignore, warning, and error. The default value is error.

This is a behavior for syntax errors in the field values in the request body. This behavior returns an error if any field value is invalid.

For semantic errors (for example, non existing ID) this header is ignored and an error is always returned.

Warnings are returned in the response headers.

Using the REST Web Services Execution Log

Each integration record includes a subtab labeled REST Web Services, under the Execution Log subtab. This log lists REST web services requests that are uniquely identified with the integration record you are currently viewing. That is, the log includes those requests that referenced the integration record's consumer key.

The execution log includes information about all requests and responses.

To see the execution log for a REST web services integration:

1. Go to Setup > Integration > Manage Integrations.
2. Select an integration record from the list.
3. On the Execution Log subtab, select REST Web Services.

For each request, the log includes the following details:

- The date and time that the request was made. This is shown in the NetSuite server time zone.
- The duration of the request.
- The status of the request.
- The email address of the user who sent the request.
- The HTTP method used and the HTTP status code
- The URL to the record or record type.
- The request and response body.

Note: The requests and responses logged in the execution log contain the values of sensitive fields in masked out format. For information about using encrypted format on custom fields, see the help topic [Encrypted Custom Field Stored Values](#).

Integration

Edit
Back
Actions

Application Id
048D1763-DF0C-44FF-83A5-6AE2452F7821

State
Enabled

Created
2023-05-11 00:00:00.0

Name
REST OAuth

Note

Created By

Description

Concurrency Limit

Last State Change
2023-05-11 00:00:00.0

Max Concurrency Limit
4

Last State Changed By

Script Id
custinteg_cce1a8ce9a56

Authentication
Execution Log
System Notes

SOAP Web Services
BEST Web Services
RESTlets

Time	Duration (Secs)	Status	User Email	Http Method	HTTP Status Code	URL	Request	Response
No records to show.								